PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE - 5th Semester
Aug – Dec 2025

UE23CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

EquiVector: A Hybrid SQL-Vector Platform for Stock Market Analytics and Semantic Company Discovery

Submitted by:

PES1UG24AM196	Pranav Achar	5D
PES1UG24AM226	Riyana Karthik	5D

Class of Prof. ***Raghu B A Rao***
Prof. Devna Sumesh

Abstract

EquiVector is a web application for stock-market analysis that keeps two kinds of data in one PostgreSQL database: structured market data (92 US-listed companies and 25,301 daily price rows from 22 September 2025 to 23 October 2026) in relational tables, and the meaning of each company's business description as a 384-dimensional embedding stored with the pgvector extension. It follows Option 2 (SQL + Vector database) of the project requirements.

The relational part has five tables with 25 named constraints, a view, three PL/pgSQL triggers, B-tree and HNSW indexes, two migrations (ALTER, DROP) and ACID transactions for every watchlist change. The vector part embeds descriptions with the all-MiniLM-L6-v2 sentence-transformer and finds similar companies by cosine distance (<=>). The project's original feature puts the two side by side: for any company it shows the five companies that describe themselves most alike and the five whose daily returns are most correlated (computed in SQL with `LAG` and `corr()`), and a report tests across all 4,186 company pairs whether semantic similarity predicts co-movement.

On the live data, description-neighbour pairs have an average return correlation of 0.297, about three times the 0.095 of random pairs but below the 0.324 of plain same-sector pairs: semantic similarity predicts co-movement mostly through the sector, not beyond it. The Next.js 15 front end talks to the database only through server-side API routes using raw, parameterised SQL. Supabase Auth provides login, and two roles (analyst: read/write, viewer: read-only) are enforced both in the API and in the database.

Table of Contents

Sl. No	Topic	Page No.
1.	Introduction a) Problem Statement	4
2.	User Requirements Specification- Tech Stack	5
3.	E-R Model a) List of Entities [labeling Weak ones] b) List non-simple Attributes c) ER Diagram Snapshot d) ERD from tool like draw.io	7
4.	Relational Model a) List of Relations that are not Entities in ER Model b) Schema Diagram [ensure light background image]	9
5.	SQL DDL Statements	11
6.	SQL DML Statements	21
7.	NoSQL/ Vector Database Commands	31
8.	Results a) Resulting Tables' Screenshots	35
9.	Next.js-Project Tree Structure	39
10.	Few sample Screenshots of application code and demo	43
11.	Evaluation Rubrics- Completion Status	51

1. Introduction

a) Problem Statement

An investor or analyst looking at one company usually asks two different questions: **which companies are similar to this one** (they do the same kind of business), and **which stocks move together with this one** (they are statistically related in the market). Today these are answered with different tools and different data. Similarity of business is judged from free-text company descriptions or from coarse labels such as "sector"; co-movement is judged from price history using correlations of returns. The two are rarely put side by side, so a third question that matters for diversification is hard to answer: **does a similar business actually mean the stocks move together?**

The two kinds of data also need different database technology. Prices, company identities, users and watchlists are structured and need exact lookups, joins, aggregates, window functions, constraints and transactions, which is the job of a relational database. Company descriptions are unstructured text: "similar meaning" cannot be found with `=` or `LIKE` ("chips" vs "semiconductors"; Nike and Ford share the same sector label), so it needs embeddings and nearest-neighbour search, which is the job of a vector database.

EquiVector stores both in PostgreSQL (with pgvector) and provides a dashboard (top movers from a SQL view, sector summaries, price history), semantic-versus-statistical company comparison, nine analytical SQL reports, per-user watchlists written in transactions with an audit trail kept by a trigger, and login with two roles. Its value is analytical rather than CRUD: window functions over 25,301 price rows, pairwise correlation of 4,186 company pairs and vector search. Source: `docs/problem-statement.md`, `docs/objectives.md`.

2. User Requirements Specification

a) Tech stack and why it fits the problem

Layer	Technology (version from the repo)	Why this choice
Web app + API	Next.js 15.5.28 (App Router, TypeScript), React 19.1, Tailwind CSS v4, shadcn/ui, Recharts	The requirements ask for a Next.js interface. API routes (`app/api/*/route.ts`) give a server-side backend layer in the same project, so database credentials never reach the browser.
Database driver	`pg` (node-postgres) ^8.23, raw SQL	Raw SQL is graded, so no ORM. Every statement lives in `db/queries/**/*.sql`, uses `$1..$n` parameters, and transactions run on one pooled client (`lib/db.ts`).
SQL database	PostgreSQL 17 hosted on Supabase	Relational data with composite keys, CHECK/FK constraints, window functions (`LAG`, `ROW_NUMBER`), `corr()`, views, triggers and ACID transactions. Supabase supplies managed Postgres with pgvector available.
Vector database	pgvector extension (0.8.2 on Supabase), `VECTOR(384)`, HNSW index	k-nearest-neighbour search on description embeddings, with the vector stored on the same row as the company, so one SQL statement can join meaning, prices and sectors.
Embeddings	sentence-transformers 6.2.0, model all-MiniLM-L6-v2 (384-dim, normalised), run locally in `seed/`	Small, free, CPU-friendly model designed for sentence similarity; normalised vectors make cosine distance a pure measure of meaning.
Data pipeline	Python: yfinance 1.7.0, SEC `company_tickers.json`, pandas, psycopg 3.3.6	Free public sources: the SEC gives the permanent CIK for each ticker; yfinance gives prices, sector, industry and description. Bulk load uses COPY into a staging table.
Authentication	Supabase Auth (`@supabase/ssr`)	Stores and hashes passwords for us (we store none). Roles live in our own `profiles` table so the database can validate them.

b) Why a vector database (Option 2) instead of a NoSQL store

The requirements allow Option 1 (SQL + NoSQL such as MongoDB or Neo4j) or Option 2 (SQL + Vector database, naming "PostgreSQL with pgvector" explicitly). EquiVector uses **Option 2**, because the technology follows from the nature of the data (`docs/database-choice-and-justification.md`):

- **Our non-relational data is not "flexible documents".** It is free text whose useful property is its meaning. MongoDB would store the text, but finding similar companies would still need embeddings and vector search; a document store adds a second system without solving the problem. A graph database fits explicit relationships, which our descriptions do not have.

- **"Which companies do similar things?" has no exact answer.** Keywords miss synonyms and the sector label is too coarse (Tesla, Ford, Nike and McDonald's are all "Consumer Cyclical"). Embeddings turn similar meaning into small cosine distance, and k-NN search with an HNSW index is exactly what a vector database provides.
- **pgvector instead of a separate vector service (Pinecone, Qdrant, Chroma):** the embedding belongs to the company row and shares its key (`cik`); the main report joins `<=>` distances with `corr()` over `prices` in one statement; when a description changes, `seed/load_data.py` resets its embedding to NULL in the same transaction; and 92 vectors x 384 floats do not justify another hosted service and credential.
- **Everything else is relational:** prices, identities, users, roles, the M:N watchlist and the audit log need constraints, joins and transactions, which a relational database does best.

3. E-R Model

a) List of entities (weak entity marked)

Entity	Type	Key	Attributes
COMPANY	Strong	cik (SEC Central Index Key)	ticker, name, sector, industry, description, description_embedding, last_updated
PRICE	**WEAK** (owner: COMPANY, identifying relationship HAS_PRICE, total participation)	Partial key trade_date; full key (cik, trade_date)	open, high, low, close, volume
PROFILE	Strong	user_id (same id as Supabase auth.users)	display_name, role, created_at
ACTIVITY_LOG	Strong	log_id (surrogate, SERIAL)	action, logged_at

Relationships: COMPANY 1:N PRICE (HAS_PRICE, identifying); PROFILE M:N COMPANY (WATCHES, with attributes note and created_at); COMPANY 1:N ACTIVITY_LOG (LOGGED_FOR, optional); PROFILE 1:N ACTIVITY_LOG (PERFORMED_BY, optional). PRICE is weak because a daily bar has no identity without its company: it is identified by the owner's key plus its own partial key trade_date, and deleting the company deletes its prices (ON DELETE CASCADE).

b) Non-simple attributes

Attribute	Kind	Notes
COMPANY.description_embedding	Array-valued (384 numbers) stored as one `VECTOR(384)` value	Not atomic in the classic sense: a fixed-length list of 384 floats. Postgres treats it as one value of the pgvector type; it is derived from the description by the embedding model. The ER diagram labels it "(VECTOR 384)" but draws a single ellipse.
PRICE (cik, trade_date)	Composite key (owner key + partial key)	trade_date alone is the partial key of the weak entity.
WATCHES (user_id, cik)	Composite key of the M:N relationship	Becomes the composite primary key of the watchlists relation.
Daily return, % change	Derived (not stored)	Computed in SQL: `close / LAG(close) - 1` (daily return) and `pct_change` in the `top_movers` view.
sector, industry, description	Optional (nullable) simple attributes	May be missing from the data source.

c) ER diagram snapshot

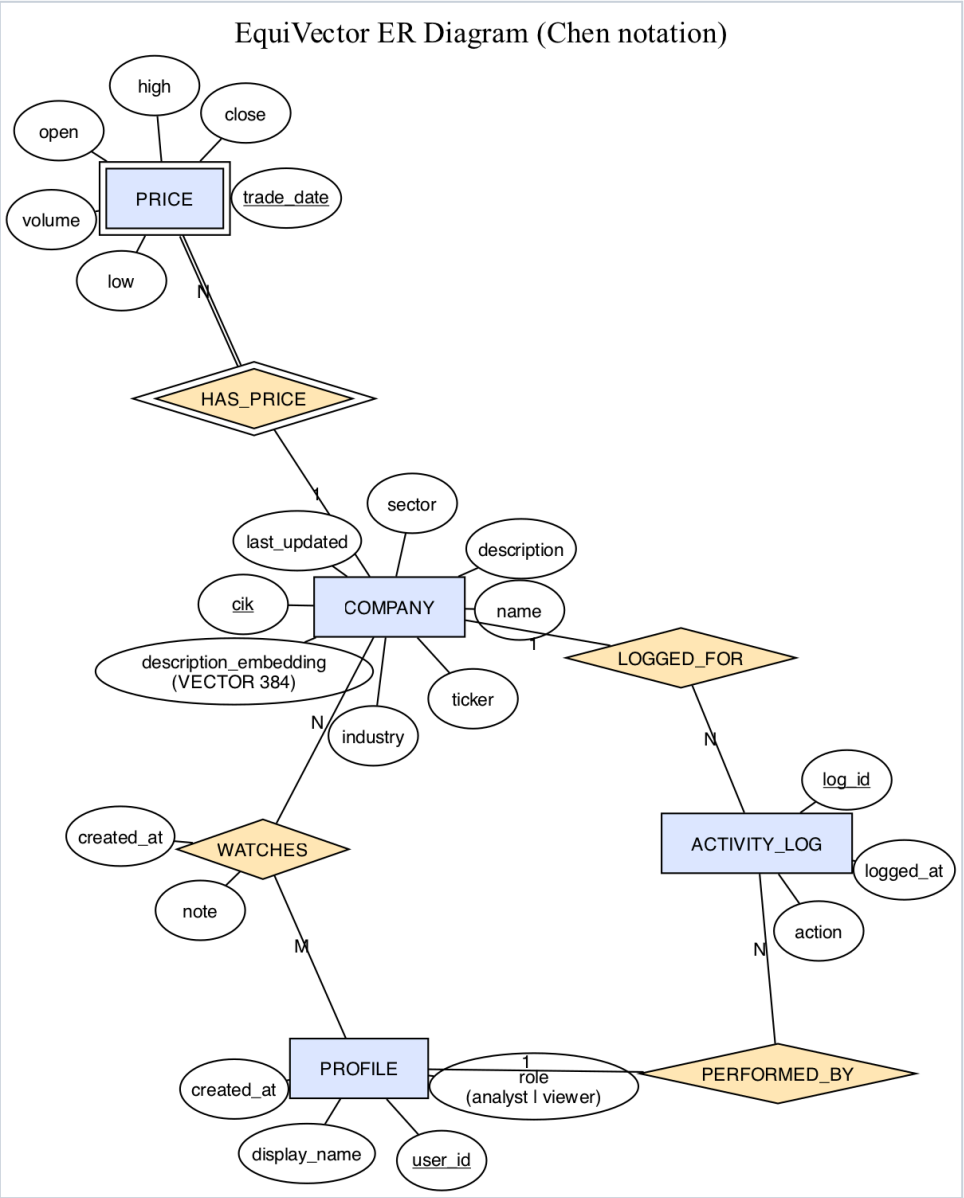


Figure 3.1: EquiVector ER diagram (Chen notation), docs/ER-diagram.png. Double border = weak entity / identifying relationship; double line = total participation; underline = key.

d) ERD from a tool

The diagram is drawn with **Graphviz** from the source file `docs/er-diagram.dot` (neato engine), not with draw.io or Lucidchart. Keeping the diagram as text means it is versioned in Git with the schema. Note: the `watchlists.updated_at` column added by migration 001 is not drawn on it.

4. Relational Model

a) List of relations (those that are not entities are labelled)

Relation	From the ER model	Primary key	Foreign keys
companies	Entity COMPANY	cik	-
prices	Weak entity PRICE (HAS_PRICE folded in as the FK)	(cik, trade_date)	cik -> companies ON DELETE CASCADE
profiles	Entity PROFILE	user_id	user_id -> auth.users(id) ON DELETE CASCADE
watchlists	**NOT an entity: the M:N relationship WATCHES** (note, created_at and, since migration 001, updated_at are relationship attributes)	(user_id, cik)	user_id -> profiles CASCADE; cik -> companies CASCADE
activity_log	Entity ACTIVITY_LOG (LOGGED_FOR and PERFORMED_BY become nullable FKs)	log_id	cik -> companies SET NULL; user_id -> profiles SET NULL
top_movers (view)	**NOT an entity:** a derived relation (SQL view over prices and companies)	-	-
auth.users	**NOT ours:** managed by Supabase Auth; referenced by profiles	id	-

b) Schema diagram

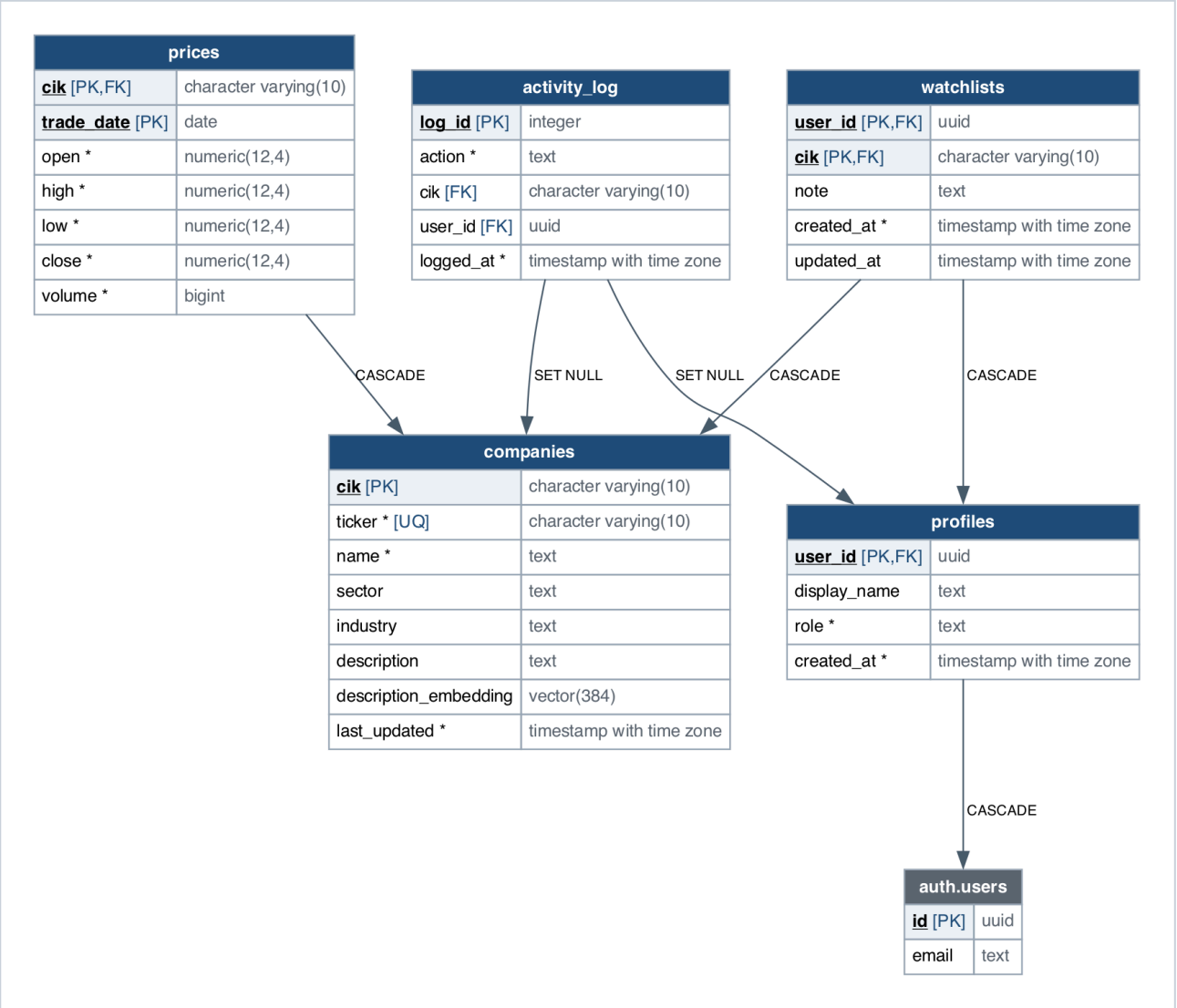


*Figure 4.1: Relational schema generated from the live database catalog (information_schema / pg_constraint) on 2026-11-09. PK = primary key, FK = foreign key, UQ = unique, * = NOT NULL; arrows go from the foreign key to the referenced key and are labelled with the ON DELETE action.*

5. SQL DDL Statements

The DDL files are run in this order: `db/schema.sql`, `indexes.sql`, `views.sql`, `triggers.sql`, then `role_enforcement.sql`, `auth_profiles.sql`, `security.sql` and the migrations. Below is a real execution log of all of them, captured on 2026-11-09 on a throwaway local PostgreSQL 17 database with pgvector 0.8.7 (`db/local/auth_stub.sql` provides the `auth.users` table that Supabase normally supplies; the throwaway database was dropped afterwards). The full source of each file follows.

Log 5.1: execution log of every DDL file (real psql output)

```
$ psql -f db/local/auth_stub.sql
CREATE SCHEMA
CREATE TABLE
$ psql -f db/schema.sql
CREATE EXTENSION
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
$ psql -f db/indexes.sql
CREATE INDEX
CREATE INDEX
$ psql -f db/views.sql
CREATE VIEW
$ psql -f db/triggers.sql
CREATE FUNCTION
CREATE TRIGGER
$ psql -f db/role_enforcement.sql
CREATE FUNCTION
CREATE TRIGGER
$ psql -f db/auth_profiles.sql
CREATE FUNCTION
CREATE TRIGGER
INSERT 0 0
$ psql -f db/security.sql
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
$ psql -f db/migrations/001_watchlists_add_updated_at.sql
BEGIN
ALTER TABLE
psql:db/migrations/001_watchlists_add_updated_at.sql:19: NOTICE:  constraint
"watchlists_updated_after_created" of relation "watchlists" does not exist, skipping
ALTER TABLE
ALTER TABLE
COMMIT
$ psql -f db/migrations/002_drop_redundant_index.sql
CREATE INDEX
--- before DROP: index sizes on prices
      index_name       |    size
-----------------------+------------
 prices_pkey           | 8192 bytes
 idx_prices_trade_date | 8192 bytes
 idx_prices_cik        | 8192 bytes
(3 rows)

DROP INDEX
--- after DROP: a per-company lookup still uses an index (prices_pkey)
                      QUERY PLAN
------------------------------------------------------
 Bitmap Heap Scan on prices
   Recheck Cond: ((cik)::text = '0000320193'::text)
   ->  Bitmap Index Scan on prices_pkey
         Index Cond: ((cik)::text = '0000320193'::text)
(4 rows)

CREATE VIEW
DROP VIEW
--- after DROP VIEW: remaining views in public
 view_name
------------
 top_movers
(1 row)
```

```
$ psql -c "\d watchlists"
                    Table "public.watchlists"
   Column   |           Type           | Collation | Nullable | Default
------------+--------------------------+-----------+----------+---------
 user_id    | uuid                     |           | not null |
 cik        | character varying(10)    |           | not null |
 note       | text                     |           |          |
 created_at | timestamp with time zone |           | not null | now()
 updated_at | timestamp with time zone |           |          |
Indexes:
    "watchlists_pkey" PRIMARY KEY, btree (user_id, cik)
Check constraints:
    "watchlists_note_len" CHECK (char_length(note) <= 500)
    "watchlists_updated_after_created" CHECK (updated_at >= created_at)
Foreign-key constraints:
    "watchlists_cik_fkey" FOREIGN KEY (cik) REFERENCES companies(cik) ON DELETE CASCADE
    "watchlists_user_id_fkey" FOREIGN KEY (user_id) REFERENCES profiles(user_id) ON DELETE CASCADE
Policies (row security enabled): (none)
Triggers:
    watchlists_analyst_only BEFORE INSERT OR DELETE OR UPDATE ON watchlists FOR EACH ROW EXECUTE FUNCTION
enforce_analyst_watchlist_writes()
```

db/schema.sql: CREATE EXTENSION and CREATE TABLE for the 5 tables, with named PK, FK, UNIQUE, CHECK and NOT NULL constraints.

```
-- ================================================================
-- EquiVector schema: 5 tables (companies, prices, profiles, watchlists,
-- activity_log). Run once against an EMPTY database, in this order:
--   1. db/schema.sql      (this file)
--   2. db/indexes.sql     (Phase 2)
--   3. db/views.sql       (Phase 2)
--   4. db/triggers.sql    (Phase 2)
--
-- Supabase: run in the SQL editor, or with
--   psql "$DATABASE_URL" -f db/schema.sql
-- Local Postgres (no Supabase): run db/local/auth_stub.sql first, because
-- profiles references auth.users, which only exists on Supabase.
--
-- Validation is done HERE, in the database (NOT NULL, CHECK, FK, UNIQUE),
-- not only in the app. Every constraint is named so that error messages
-- in the app and in the test log say which rule was broken.
-- ================================================================

-- pgvector adds the VECTOR(n) column type, the <=> (cosine distance)
-- operator and the HNSW index type.
CREATE EXTENSION IF NOT EXISTS vector;

-- ----------------------------------------------------------------
-- 1. companies: one row per listed company.
--    Structured identity/classification columns are plain SQL; the free
--    text description is also stored as a 384-dim embedding (pgvector).
-- ----------------------------------------------------------------
CREATE TABLE companies (
    cik                 VARCHAR(10)  PRIMARY KEY,   -- SEC Central Index Key, zero-padded
    ticker              VARCHAR(10)  NOT NULL,
    name                TEXT         NOT NULL,
    sector              TEXT,
    industry            TEXT,
    description         TEXT,                       -- source text for the embedding
    description_embedding VECTOR(384),              -- all-MiniLM-L6-v2, L2-normalised
    last_updated        TIMESTAMPTZ  NOT NULL DEFAULT now(),

    CONSTRAINT companies_ticker_key   UNIQUE (ticker),
    -- CIKs are always 10 digits once zero-padded (e.g. 0000320193 = Apple)
    CONSTRAINT companies_cik_format   CHECK (cik ~ '^[0-9]{10}$'),
    -- upper-case tickers; '.' and '-' allowed for share classes (BRK-B)
    CONSTRAINT companies_ticker_format CHECK (ticker ~ '^[A-Z0-9.-]{1,10}$'),
    CONSTRAINT companies_name_not_blank CHECK (btrim(name) <> '')
);

-- ----------------------------------------------------------------
-- 2. prices: daily OHLCV bars. Weak entity of companies: a bar is
--    identified by (cik, trade_date), so the PK is composite and there
--    is no surrogate id.
-- ----------------------------------------------------------------
CREATE TABLE prices (
    cik         VARCHAR(10)    NOT NULL,
    trade_date  DATE           NOT NULL,
    open        NUMERIC(12,4)  NOT NULL,
    high        NUMERIC(12,4)  NOT NULL,
```

```
    low         NUMERIC(12,4)  NOT NULL,
    close       NUMERIC(12,4)  NOT NULL,
    volume      BIGINT         NOT NULL,

    CONSTRAINT prices_pkey PRIMARY KEY (cik, trade_date),
    -- deleting a company deletes its price history (identifying relationship)
    CONSTRAINT prices_cik_fkey FOREIGN KEY (cik)
        REFERENCES companies (cik) ON DELETE CASCADE,
    CONSTRAINT prices_positive     CHECK (open > 0 AND high > 0 AND low > 0 AND close > 0),
    CONSTRAINT prices_volume_nonneg CHECK (volume >= 0),
    -- a bar's high must be the highest price of the day, its low the lowest
    CONSTRAINT prices_high_is_max  CHECK (high >= GREATEST(open, close, low)),
    CONSTRAINT prices_low_is_min   CHECK (low  <= LEAST(open, close)),
    -- no weekend bars (US exchanges are closed Sat/Sun); ISODOW 6 = Sat, 7 = Sun
    CONSTRAINT prices_trading_day  CHECK (EXTRACT(ISODOW FROM trade_date) < 6)
);

-- ---------------------------------------------------------------------
-- 3. profiles: app-level user record, 1:1 with Supabase's auth.users.
--    Supabase Auth stores the password hash; we store NO credentials.
--    The role lives here (not in auth metadata) so the database itself
--    can validate it with a CHECK constraint.
-- ---------------------------------------------------------------------
CREATE TABLE profiles (
    user_id       UUID         PRIMARY KEY,
    display_name  TEXT,
    role          TEXT         NOT NULL DEFAULT 'viewer',
    created_at    TIMESTAMPTZ  NOT NULL DEFAULT now(),

    CONSTRAINT profiles_user_id_fkey FOREIGN KEY (user_id)
        REFERENCES auth.users (id) ON DELETE CASCADE,
    CONSTRAINT profiles_role_check CHECK (role IN ('analyst', 'viewer')),
    CONSTRAINT profiles_display_name_len CHECK (char_length(display_name) <= 60)
);

-- ---------------------------------------------------------------------
-- 4. watchlists: the M:N WATCHES relationship between profiles and
--    companies. note and created_at are attributes of the relationship.
--    The composite PK means a user cannot watch the same company twice.
-- ---------------------------------------------------------------------
CREATE TABLE watchlists (
    user_id     UUID         NOT NULL,
    cik         VARCHAR(10)  NOT NULL,
    note        TEXT,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),

    CONSTRAINT watchlists_pkey PRIMARY KEY (user_id, cik),
    CONSTRAINT watchlists_user_id_fkey FOREIGN KEY (user_id)
        REFERENCES profiles (user_id) ON DELETE CASCADE,
    CONSTRAINT watchlists_cik_fkey FOREIGN KEY (cik)
        REFERENCES companies (cik) ON DELETE CASCADE,
    CONSTRAINT watchlists_note_len CHECK (char_length(note) <= 500)
);

-- ---------------------------------------------------------------------
-- 5. activity_log: audit trail. Written by the AFTER INSERT ON prices
--    trigger (user_id NULL) and by the watchlist transaction (user_id set).
--    FKs use ON DELETE SET NULL so history survives deleted users/companies.
-- ---------------------------------------------------------------------
CREATE TABLE activity_log (
    log_id     SERIAL       PRIMARY KEY,
    action     TEXT         NOT NULL,
    cik        VARCHAR(10),
    user_id    UUID,
    logged_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),

    CONSTRAINT activity_log_cik_fkey FOREIGN KEY (cik)
        REFERENCES companies (cik) ON DELETE SET NULL,
    CONSTRAINT activity_log_user_id_fkey FOREIGN KEY (user_id)
        REFERENCES profiles (user_id) ON DELETE SET NULL,
    -- closed set of actions, so the "summary per action type" report is clean
    CONSTRAINT activity_log_action_check CHECK (action IN (
        'PRICE_INSERT',       -- prices trigger
        'WATCHLIST_ADD',      -- watchlist transaction
        'WATCHLIST_UPDATE',   -- note edited
        'WATCHLIST_REMOVE'    -- entry deleted
    ))
);
```

db/indexes.sql: CREATE INDEX: B-tree on prices(trade_date) and HNSW on the description embeddings.

```
-- ========================================================================
-- Indexes. Run after db/schema.sql:  psql "$DATABASE_URL" -f db/indexes.sql
--
-- Indexes that ALREADY exist automatically (not repeated here):
--   companies_pkey (cik), companies_ticker_key (ticker)
--   prices_pkey (cik, trade_date): also serves "all prices of one company"
--     because cik is its leading column; a separate index on prices(cik)
--     would be redundant (see db/migrations/002_drop_redundant_index.sql).
--   watchlists_pkey (user_id, cik), profiles_pkey, activity_log_pkey
-- ========================================================================

-- 1. B-tree on trade_date.
--    The composite PK starts with cik, so it cannot help queries that ask
--    "what are the latest trading dates across ALL companies?" (the first
--    step of the top_movers view) or "all prices on date X". This index lets
--    Postgres read dates newest-first straight from the index instead of
--    scanning and sorting the whole prices table.
CREATE INDEX IF NOT EXISTS idx_prices_trade_date
    ON prices (trade_date);

-- 2. HNSW index on the description embeddings (pgvector).
--    HNSW = Hierarchical Navigable Small World: a layered graph in which
--    each vector links to its near neighbours, so a nearest-neighbour search
--    hops through the graph instead of comparing against every row.
--    It is approximate (ANN): very fast, may very occasionally miss a true
--    neighbour.
--    The operator class MUST match the operator used in queries:
--      vector_cosine_ops  <->  ORDER BY description_embedding <=> $query
--    (a vector_l2_ops index would be ignored by a <=> query).
--    m / ef_construction are pgvector's defaults, written out for clarity:
--      m = 16              links per node per layer
--      ef_construction = 64 candidate list size while building
--    With 92 rows the planner may still choose a sequential scan (cheaper at
--    this size); the index is what keeps the search fast as the table grows.
CREATE INDEX IF NOT EXISTS idx_companies_embedding
    ON companies USING hnsw (description_embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);
```

db/views.sql: CREATE VIEW top_movers (two CTEs and a self-join of prices).

```
-- ========================================================================
-- Views. Run after db/indexes.sql:  psql "$DATABASE_URL" -f db/views.sql
-- ========================================================================

-- top_movers: % change of every company between the two most recent
-- trading days in the database, biggest gainers first.
--
-- A view stores the QUERY, not the result, so it is always current:
-- after load_data.py --batch 2 inserts a new week, the same
-- SELECT * FROM top_movers automatically compares the new latest days.
CREATE OR REPLACE VIEW top_movers AS
WITH latest_two_days AS (
    -- The two most recent trading dates across all companies.
    -- Served by idx_prices_trade_date (read newest-first, stop after 2).
    SELECT DISTINCT trade_date
    FROM prices
    ORDER BY trade_date DESC
    LIMIT 2
),
day_pair AS (
    -- Turn the two dates into one row: (latest day, the day before it).
    SELECT max(trade_date) AS cur_date,
           min(trade_date) AS prev_date
    FROM latest_two_days
)
SELECT
    c.cik,
    c.ticker,
    c.name,
    c.sector,
    d.prev_date,
    prev.close                                      AS prev_close,
    d.cur_date                                      AS trade_date,
    cur.close                                       AS close,
    -- percentage change = (today - yesterday) / yesterday * 100
    round((cur.close - prev.close) / prev.close * 100, 2) AS pct_change,
    cur.volume
FROM day_pair d
```

```
JOIN prices cur                      -- each company's bar on the latest day
  ON cur.trade_date = d.cur_date
JOIN prices prev                     -- the same company's bar on the previous day
  ON prev.cik = cur.cik
 AND prev.trade_date = d.prev_date
JOIN companies c                     -- add ticker/name/sector for display
  ON c.cik = cur.cik
ORDER BY pct_change DESC;
```

db/triggers.sql: AFTER INSERT ON prices trigger and its PL/pgSQL function (audit log and last_updated).

```
-- ====================================================================
-- Triggers. Run after db/views.sql:  psql "$DATABASE_URL" -f db/triggers.sql
-- ====================================================================

-- Trigger function: runs once for EVERY row inserted into prices.
-- NEW is the row that was just inserted.
CREATE OR REPLACE FUNCTION trg_prices_after_insert()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- 1. Record that this company received fresh market data.
    UPDATE companies
       SET last_updated = now()
     WHERE cik = NEW.cik;

    -- 2. Write an audit row. user_id stays NULL: the insert came from the
    --    seed pipeline, not from a logged-in user.
    INSERT INTO activity_log (action, cik)
    VALUES ('PRICE_INSERT', NEW.cik);

    -- The return value of an AFTER trigger is ignored; NULL is conventional.
    RETURN NULL;
END;
$$;

-- AFTER INSERT: fires only once the price row has passed all CHECK/FK
-- constraints and been written. Because it runs inside the same
-- transaction as the INSERT, a failed batch rolls back the log rows and
-- last_updated changes too: they can never disagree with the prices table.
CREATE OR REPLACE TRIGGER prices_after_insert
AFTER INSERT ON prices
FOR EACH ROW
EXECUTE FUNCTION trg_prices_after_insert();
```

db/role_enforcement.sql: BEFORE INSERT/UPDATE/DELETE ON watchlists trigger: only analysts may write.

```
-- ====================================================================
-- Database-level role enforcement: only ANALYSTS may have watchlist
-- rows written for them. Second line of defence behind the API's 403.
--   psql "$DATABASE_URL" -f db/role_enforcement.sql
--
-- Why a trigger and not a CHECK: a CHECK constraint can only look at the
-- row being written, but the role lives in another table (profiles).
-- A BEFORE trigger can query profiles and reject the write.
-- (profiles.role itself stays validated by profiles_role_check.)
--
-- Note: the app connects as one database user, so the database checks the
-- role of the user the row BELONGS to (watchlists.user_id), which the API
-- always sets to the logged-in user's id.
-- ====================================================================
CREATE OR REPLACE FUNCTION enforce_analyst_watchlist_writes()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP IN ('INSERT', 'UPDATE') THEN
        -- the row's owner must exist as an analyst
        IF NOT EXISTS (SELECT 1 FROM profiles
                       WHERE user_id = NEW.user_id AND role = 'analyst') THEN
            RAISE EXCEPTION 'only analysts can modify watchlists (user %)', NEW.user_id
                USING ERRCODE = '42501';           -- insufficient_privilege -> API returns
403
        END IF;
        RETURN NEW;
    END IF;

    -- DELETE: block a viewer. If the profile no longer exists (the account is
    -- being deleted and the delete is CASCADING here), allow it.
```

```
    IF EXISTS (SELECT 1 FROM profiles
               WHERE user_id = OLD.user_id AND role <> 'analyst') THEN
        RAISE EXCEPTION 'only analysts can modify watchlists (user %)', OLD.user_id
            USING ERRCODE = '42501';
    END IF;
    RETURN OLD;
END;
$$;

-- BEFORE: runs before the row is written, so RAISE stops the write and
-- (inside the app's transaction) rolls back the activity_log insert too.
CREATE OR REPLACE TRIGGER watchlists_analyst_only
BEFORE INSERT OR UPDATE OR DELETE ON watchlists
FOR EACH ROW
EXECUTE FUNCTION enforce_analyst_watchlist_writes();
```

db/auth_profiles.sql: AFTER INSERT ON auth.users trigger: every signup gets a 'viewer' profile, plus a one-off backfill.

```
-- ============================================================================
-- Auth -> profiles: every new Supabase Auth user automatically gets a
-- row in public.profiles with the default role 'viewer'.
--   psql "$DATABASE_URL" -f db/auth_profiles.sql
-- Safe to re-run (CREATE OR REPLACE, ON CONFLICT DO NOTHING).
-- ============================================================================

-- Trigger function. Runs once for every row inserted into auth.users
-- (i.e. every signup). NEW is the new auth user.
--
-- SECURITY DEFINER: the function runs with the privileges of its OWNER
-- (postgres), not of the caller. Signups are inserted by Supabase's
-- internal auth role, which has no rights on public.profiles; this lets
-- the insert succeed anyway.
-- SET search_path = '': because the function runs with elevated rights,
-- it must not resolve names through a search_path someone else could
-- influence (a malicious schema could shadow "profiles"). With an empty
-- search_path every name must be schema-qualified (public.profiles), so
-- the function can only ever touch the intended table.
CREATE OR REPLACE FUNCTION public.handle_new_auth_user()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = ''
AS $$
BEGIN
    INSERT INTO public.profiles (user_id, display_name, role)
    VALUES (
        NEW.id,
        left(split_part(NEW.email, '@', 1), 60),  -- "analyst@gmail.com" -> "analyst" (<= 60
chars, see CHECK)
        'viewer'                                  -- least privilege by default; promote with
UPDATE
    )
    ON CONFLICT (user_id) DO NOTHING;             -- never fail a signup because a profile
already exists
    RETURN NEW;                                   -- ignored for AFTER triggers
END;
$$;

-- AFTER INSERT: the auth.users row exists by the time we insert the
-- profile, so the profiles.user_id -> auth.users(id) foreign key is satisfied.
CREATE OR REPLACE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW
EXECUTE FUNCTION public.handle_new_auth_user();

-- Backfill: users who signed up BEFORE this trigger existed have no
-- profile yet. Give each of them a 'viewer' profile now.
INSERT INTO public.profiles (user_id, display_name, role)
SELECT u.id,
       left(split_part(u.email, '@', 1), 60),
       'viewer'
FROM auth.users u
WHERE NOT EXISTS (SELECT 1 FROM public.profiles p WHERE p.user_id = u.id);
```

db/security.sql: ALTER TABLE ... ENABLE ROW LEVEL SECURITY: blocks Supabase's public REST API.

```
-- ============================================================================
-- Lock the tables away from Supabase's auto-generated REST API.
```

```
-- Run after db/schema.sql:  psql "$DATABASE_URL" -f db/security.sql
--
-- Why: Supabase publishes every table in the `public` schema through its
-- REST API (PostgREST) to the `anon` and `authenticated` roles. Their key
-- (NEXT_PUBLIC_SUPABASE_ANON_KEY) is visible in the browser, so without
-- this anyone could read or write our tables directly, bypassing the API
-- routes and their role checks.
--
-- Enabling Row Level Security with NO policies denies every row to those
-- roles. Our app connects through `pg` as the table owner (postgres), and
-- owners are not subject to RLS, so the app's own queries are unaffected.
-- All data access therefore goes through the Next.js API layer.
-- ======================================================================

ALTER TABLE companies    ENABLE ROW LEVEL SECURITY;
ALTER TABLE prices       ENABLE ROW LEVEL SECURITY;
ALTER TABLE profiles     ENABLE ROW LEVEL SECURITY;
ALTER TABLE watchlists   ENABLE ROW LEVEL SECURITY;
ALTER TABLE activity_log ENABLE ROW LEVEL SECURITY;
```

db/migrations/001_watchlists_add_updated_at.sql: Migration 001 (ALTER TABLE): add updated_at and a CHECK constraint.

```
-- ======================================================================
-- Migration 001: ALTER TABLE, evolve a live table without recreating it.
--   psql "$DATABASE_URL" -f db/migrations/001_watchlists_add_updated_at.sql
--
-- Why: Phase 3 lets an analyst edit a watchlist note (an UPDATE). We want
-- to record WHEN the note was last edited, separately from created_at.
-- Safe to re-run (IF NOT EXISTS / DROP ... IF EXISTS).
-- ======================================================================

BEGIN;

-- 1. ADD COLUMN: nullable, because existing rows have never been edited.
ALTER TABLE watchlists
    ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ;

-- 2. ADD CONSTRAINT: an edit cannot happen before the row was created.
--    (NULL passes a CHECK, so un-edited rows are fine.)
ALTER TABLE watchlists
    DROP CONSTRAINT IF EXISTS watchlists_updated_after_created;
ALTER TABLE watchlists
    ADD CONSTRAINT watchlists_updated_after_created
    CHECK (updated_at >= created_at);

COMMIT;
```

db/migrations/002_drop_redundant_index.sql: Migration 002 (DROP INDEX, DROP VIEW): remove a redundant index and a scratch view.

```
-- ======================================================================
-- Migration 002: DROP, removing a redundant index (and a throwaway view).
--   psql "$DATABASE_URL" -f db/migrations/002_drop_redundant_index.sql
--
-- Story: an early draft of the design indexed prices(cik) to speed up
-- "all prices of one company". That index is redundant: the primary key
-- index prices_pkey is on (cik, trade_date), and a B-tree on (a, b) can
-- serve lookups on its leading column `a` alone. A redundant index costs
-- disk space and slows every INSERT (both indexes must be updated, which
-- matters for the price batches) while giving no read benefit.
--
-- This script re-creates that draft index so the demo is self-contained,
-- shows its cost, then DROPs it and shows the query still uses an index.
-- ======================================================================

-- The draft index (as it would have existed before this migration).
CREATE INDEX IF NOT EXISTS idx_prices_cik ON prices (cik);

\echo '--- before DROP: index sizes on prices'
SELECT indexrelid::regclass AS index_name,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_index
WHERE indrelid = 'prices'::regclass
ORDER BY 1;

-- The actual migration.
DROP INDEX IF EXISTS idx_prices_cik;
```

```
\echo '--- after DROP: a per-company lookup still uses an index (prices_pkey)'
EXPLAIN (COSTS OFF)
SELECT trade_date, close FROM prices WHERE cik = '0000320193';

-- DROP VIEW: a scratch view created while prototyping top_movers, no
-- longer needed now that the real view exists in db/views.sql.
CREATE OR REPLACE VIEW scratch_latest_close AS
SELECT cik, max(trade_date) AS latest_date FROM prices GROUP BY cik;
DROP VIEW IF EXISTS scratch_latest_close;

\echo '--- after DROP VIEW: remaining views in public'
SELECT table_name AS view_name
FROM information_schema.views
WHERE table_schema = 'public';
```

6. SQL DML Statements

Every SQL statement the app runs is a file in `db/queries/` (one statement per file, `$1..$n` parameters), loaded with `loadSql()` and executed through the `pg` driver. The seed scripts in `seed/` run their own statements through psycopg. Statements are grouped by type below.

a) INSERT

db/queries/watchlist/insert.sql: step 1 of the add-to-watchlist transaction

```
-- Step 1 of the "add to watchlist" TRANSACTION (POST /api/watchlist).
--   $1 = user_id, $2 = cik, $3 = note
-- Duplicate (user, company) -> violates watchlists_pkey -> whole transaction rolls back.
INSERT INTO watchlists (user_id, cik, note)
VALUES ($1, $2, $3)
RETURNING user_id, cik, note, created_at;
```

db/queries/watchlist/log_action.sql: step 2 of every watchlist transaction (audit row)

```
-- Step 2 of every watchlist TRANSACTION: audit row, written by the SAME
-- transaction as the change it records, so both commit or neither does.
--   $1 = action ('WATCHLIST_ADD' | 'WATCHLIST_UPDATE' | 'WATCHLIST_REMOVE'), $2 = cik, $3 =
user_id
INSERT INTO activity_log (action, cik, user_id)
VALUES ($1, $2, $3)
RETURNING log_id;
```

seed/load_data.py: INSERT ... ON CONFLICT DO UPDATE (upsert of the 92 companies)

```
# Upsert: insert new companies; for existing ones refresh the text columns.
# If the description text changed, the old embedding no longer matches it,
# so it is reset to NULL and embed_descriptions.py will recompute it.
UPSERT_COMPANY = """
```

seed/load_data.py: bulk load with COPY into a temporary staging table, then INSERT ... SELECT ... ON CONFLICT DO NOTHING (fires the prices trigger)

```
# Prices are bulk-loaded with COPY into a temporary staging table (fast over
# the network), then moved into the real table with INSERT ... SELECT so that
# all constraints and triggers on `prices` apply, and rows that already exist
# are skipped rather than failing the whole batch.
CREATE_STAGING = "CREATE TEMP TABLE staging_prices (LIKE prices INCLUDING DEFAULTS) ON COMMIT
DROP"
COPY_STAGING = "COPY staging_prices (cik, trade_date, open, high, low, close, volume) FROM
STDIN"
MOVE_STAGING = """
INSERT INTO prices (cik, trade_date, open, high, low, close, volume)
SELECT cik, trade_date, open, high, low, close, volume
FROM staging_prices
ON CONFLICT (cik, trade_date) DO NOTHING
"""
```

b) UPDATE

db/queries/watchlist/update_note.sql: edit a watchlist note

```
-- UPDATE: edit a watchlist note (PATCH /api/watchlist).
--   $1 = user_id, $2 = cik, $3 = new note
-- updated_at was added by db/migrations/001 (ALTER TABLE); its CHECK
-- constraint guarantees updated_at >= created_at.
UPDATE watchlists
SET note = $3,
    updated_at = now()
WHERE user_id = $1
  AND cik = $2
RETURNING cik, note, updated_at;
```

seed/embed_descriptions.py: store the embeddings (UPDATE ... SET description_embedding = %s::vector)

```
cur.executemany(
    "UPDATE companies SET description_embedding = %s::vector WHERE cik = %s",
    [(to_pgvector(v), cik) for (cik, _), v in zip(rows, vectors)],
)
```

db/admin/set_role.sql: promote a user (admin task, run from psql only)

```
-- ======================================================================
-- Change a user's role (admin task, run from psql, never from the app).
--   psql "$DATABASE_URL" -v email=analyst@gmail.com -v role=analyst -f db/admin/set_role.sql
-- profiles_role_check rejects anything other than 'analyst' / 'viewer'.
-- ======================================================================
\set ON_ERROR_STOP on

UPDATE profiles
SET role = :'role'
WHERE user_id = (SELECT id FROM auth.users WHERE email = :'email')
RETURNING user_id, display_name, role;
```

c) DELETE

db/queries/watchlist/delete.sql: remove a company from the logged-in user's watchlist

```
-- DELETE: remove a company from the user's watchlist (DELETE /api/watchlist).
--   $1 = user_id, $2 = cik
DELETE FROM watchlists
WHERE user_id = $1
  AND cik = $2
RETURNING cik;
```

d) SELECT used by the app

db/queries/companies_search.sql

```
-- Company search + filter for the UI (GET /api/companies?q=&sector=).
--   $1 = search text or NULL (matches ticker prefix or any part of the name)
--   $2 = sector or NULL
-- ILIKE = case-insensitive LIKE. "$1::text IS NULL OR ..." makes each filter
-- optional with one fixed, parameterised statement (no SQL string building).
SELECT cik, ticker, name, sector, industry
FROM companies
WHERE ($1::text IS NULL OR ticker ILIKE $1 || '%' OR name ILIKE '%' || $1 || '%')
  AND ($2::text IS NULL OR sector = $2)
ORDER BY ticker;
```

db/queries/company_by_ticker.sql

```
-- One company's details (header of the /search comparison).  $1 = ticker
SELECT cik, ticker, name, sector, industry, description
FROM companies
WHERE ticker = $1;
```

db/queries/prices_for_company.sql

```
-- Daily price history of one company for the dashboard chart
-- (GET /api/prices?ticker=).  $1 = ticker
SELECT p.trade_date, p.open, p.high, p.low, p.close, p.volume
FROM prices p
JOIN companies c ON c.cik = p.cik
WHERE c.ticker = $1
ORDER BY p.trade_date;      -- served by prices_pkey (cik, trade_date)
```

db/queries/top_movers.sql

```
-- Full top_movers view for the dashboard (GET /api/top-movers).
-- The view (db/views.sql) compares the two latest trading days.
SELECT ticker, name, sector, prev_date, trade_date, prev_close, close, pct_change, volume
FROM top_movers;
```

db/queries/watchlist/list.sql

```
-- The user's watchlist with each company's latest close (GET /api/watchlist).
--   $1 = user_id
SELECT c.ticker, c.name, c.sector, w.cik, w.note, w.created_at, w.updated_at,
       latest.close AS latest_close, latest.trade_date AS latest_date
FROM watchlists w
JOIN companies c ON c.cik = w.cik
-- LATERAL: for each watchlist row, fetch that company's most recent price
CROSS JOIN LATERAL (
    SELECT p.close, p.trade_date
    FROM prices p
    WHERE p.cik = w.cik
    ORDER BY p.trade_date DESC
    LIMIT 1
) AS latest
WHERE w.user_id = $1
ORDER BY w.created_at DESC;
```

db/queries/auth/profile_by_user.sql

```
-- The logged-in user's profile, looked up by the id Supabase Auth verified
-- from the session cookie (never by anything the client sends).
--   $1 = user_id (auth.users.id)
SELECT user_id, display_name, role
FROM profiles
WHERE user_id = $1;
```

db/queries/correlated_companies.sql

```
-- ========================================================================
-- STATISTICAL similarity: top-5 companies whose daily returns are most
-- correlated with one company's, over the last year.
--   $1 = ticker, e.g. 'JPM'
--   npx tsx scripts/run-sql.ts db/queries/correlated_companies.sql JPM
--
-- Daily return on day t = close(t) / close(t-1) - 1.
-- Pearson correlation = corr(x, y), +1 = always move together,
-- 0 = unrelated, -1 = always move in opposite directions.
-- ========================================================================
WITH last_year AS (
    -- Step 1: only the most recent year of prices.
    SELECT cik, trade_date, close
    FROM prices
    WHERE trade_date > (SELECT max(trade_date) FROM prices) - INTERVAL '1 year'
),
daily_returns AS (
    -- Step 2: LAG(close) = the previous row's close for the SAME company
    -- (PARTITION BY cik) in date order (ORDER BY trade_date).
    -- Each company's first day has no previous close, so ret is NULL there;
    -- corr() and regr_count() skip NULL pairs automatically.
    SELECT cik,
           trade_date,
           (close / LAG(close) OVER (PARTITION BY cik ORDER BY trade_date) - 1)::float8 AS ret
    FROM last_year
)
-- Step 3: pair the target's return with every other company's return ON THE
-- SAME DAY (join on trade_date = "aligned" returns), then correlate per company.
SELECT c.ticker,
       c.name,
       c.sector,
       c.industry,
       round(corr(t.ret, o.ret)::numeric, 3) AS return_correlation,
       regr_count(t.ret, o.ret)              AS days_used   -- non-NULL pairs
FROM daily_returns t                         -- t = target company
JOIN daily_returns o                         -- o = every other company
  ON o.trade_date = t.trade_date
 AND o.cik <> t.cik
JOIN companies c
  ON c.cik = o.cik
WHERE t.cik = (SELECT cik FROM companies WHERE ticker = $1)
GROUP BY c.ticker, c.name, c.sector, c.industry
ORDER BY return_correlation DESC NULLS LAST
LIMIT 5;
```

e) Report queries (JOIN, GROUP BY, HAVING, CTE, subqueries, window functions)

db/queries/reports/sector_summary.sql: JOIN, GROUP BY, aggregates, ORDER BY

```
-- ====================================================================
-- REPORT: Average closing price and average daily volume per sector.
-- Demonstrates: JOIN + GROUP BY + aggregate functions + ORDER BY.
--   npx tsx scripts/run-sql.ts db/queries/reports/sector_summary.sql
-- Covers every price row loaded (2025-09-22 .. latest).
-- ====================================================================
SELECT c.sector,
       count(DISTINCT c.cik)           AS companies,     -- distinct: each company has ~275
rows
       round(avg(p.close), 2)          AS avg_close,
       round(avg(p.volume))            AS avg_daily_volume,
       min(p.trade_date)               AS from_date,
       max(p.trade_date)               AS to_date
FROM companies c
JOIN prices p                         -- one output row per (company, day) before grouping
  ON p.cik = c.cik
GROUP BY c.sector                     -- collapse to one row per sector
ORDER BY avg_daily_volume DESC;       -- most heavily traded sectors first
```

db/queries/reports/sector_having.sql: CTE (WITH), self-join, GROUP BY, HAVING

```
-- ====================================================================
-- REPORT: Sectors with more than N companies whose average 1-year
-- % change is above X.
-- Demonstrates: GROUP BY + HAVING (a filter on GROUPS, applied after
-- aggregation; WHERE filters rows before aggregation).
--   $1 = N (min companies, exclusive), $2 = X (min avg % change, exclusive)
--   npx tsx scripts/run-sql.ts db/queries/reports/sector_having.sql 6 0
-- ====================================================================
WITH window_bounds AS (
    -- each company's first and last trading day within the last year
    SELECT cik,
           min(trade_date) AS first_day,
           max(trade_date) AS last_day
    FROM prices
    WHERE trade_date > (SELECT max(trade_date) FROM prices) - INTERVAL '1 year'
    GROUP BY cik
),
company_change AS (
    -- % change from the first to the last close in that window
    SELECT b.cik,
           (p_last.close / p_first.close - 1) * 100 AS pct_change
    FROM window_bounds b
    JOIN prices p_first ON p_first.cik = b.cik AND p_first.trade_date = b.first_day
    JOIN prices p_last  ON p_last.cik  = b.cik AND p_last.trade_date  = b.last_day
)
SELECT c.sector,
       count(*)                       AS companies,
       round(avg(cc.pct_change), 2)   AS avg_pct_change_1y,
       round(min(cc.pct_change), 2)   AS worst_company_pct,
       round(max(cc.pct_change), 2)   AS best_company_pct
FROM company_change cc
JOIN companies c ON c.cik = cc.cik
GROUP BY c.sector
HAVING count(*) > $1                  -- keep only sectors with more than N companies
   AND avg(cc.pct_change) > $2        -- ...and an average % change above X
ORDER BY avg_pct_change_1y DESC;
```

db/queries/reports/above_own_average.sql: correlated subqueries in SELECT, JOIN and WHERE

```
-- ====================================================================
-- REPORT: Companies whose latest close is above their OWN average close.
-- Demonstrates: correlated SUBQUERIES. The inner SELECTs refer to the
-- outer row (c.cik), so they are re-evaluated for each company.
--   npx tsx scripts/run-sql.ts db/queries/reports/above_own_average.sql
-- ====================================================================
SELECT c.ticker,
       c.name,
       c.sector,
       p.close                                                AS latest_close,
       round((SELECT avg(p2.close)
              FROM prices p2
              WHERE p2.cik = c.cik), 2)                       AS avg_close,
       round((p.close / (SELECT avg(p2.close)
```

```
                         FROM prices p2
                         WHERE p2.cik = c.cik) - 1) * 100, 2) AS pct_above_avg
FROM companies c
JOIN prices p
  ON p.cik = c.cik
 -- subquery 1: pick only this company's most recent price row
 AND p.trade_date = (SELECT max(p3.trade_date)
                     FROM prices p3
                     WHERE p3.cik = c.cik)
-- subquery 2: keep the company only if that close beats its own average
WHERE p.close > (SELECT avg(p2.close)
                 FROM prices p2
                 WHERE p2.cik = c.cik)
ORDER BY pct_above_avg DESC;
```

db/queries/reports/top_movers_top10.sql: querying a view

```
-- ========================================================================
-- REPORT: Top 10 movers on the latest trading day.
-- Demonstrates: querying a VIEW (db/views.sql) like a table.
--   npx tsx scripts/run-sql.ts db/queries/reports/top_movers_top10.sql
-- The view is already ordered by pct_change DESC; LIMIT keeps the top 10.
-- ========================================================================
SELECT ticker,
       name,
       sector,
       prev_date,
       trade_date,
       prev_close,
       close,
       pct_change,
       volume
FROM top_movers
LIMIT 10;
```

db/queries/reports/most_watched.sql: JOIN across the M:N table, GROUP BY, COUNT

```
-- ========================================================================
-- REPORT: Most-watched companies across all users.
-- Demonstrates: JOIN across the M:N relationship table + GROUP BY + COUNT.
--   npx tsx scripts/run-sql.ts db/queries/reports/most_watched.sql
-- ========================================================================
SELECT c.ticker,
       c.name,
       c.sector,
       count(w.user_id)   AS watchers,        -- how many users watch it
       max(w.created_at)  AS last_added
FROM watchlists w
JOIN companies c
  ON c.cik = w.cik
GROUP BY c.ticker, c.name, c.sector
ORDER BY watchers DESC, c.ticker
LIMIT 10;
```

db/queries/reports/activity_summary.sql: GROUP BY with COUNT, COUNT(DISTINCT), MIN, MAX

```
-- ========================================================================
-- REPORT: Activity log summary per action type.
-- Demonstrates: GROUP BY + several aggregates, COUNT(DISTINCT ...).
-- PRICE_INSERT rows come from the prices trigger (no user);
-- WATCHLIST_* rows come from the app's transactions (with user_id).
--   npx tsx scripts/run-sql.ts db/queries/reports/activity_summary.sql
-- ========================================================================
SELECT action,
       count(*)                  AS events,
       count(DISTINCT cik)       AS companies,
       count(DISTINCT user_id)   AS users,     -- COUNT(DISTINCT) ignores NULLs
       min(logged_at)            AS first_at,
       max(logged_at)            AS last_at
FROM activity_log
GROUP BY action
ORDER BY events DESC;
```

db/queries/reports/semantic_vs_statistical.sql: CTEs, LAG window function, corr(), CROSS JOIN LATERAL, UNION ALL, pgvector <=>

```
-- ========================================================================
-- REPORT: Does semantic similarity predict return correlation?
```

```
--   npx tsx scripts/run-sql.ts db/queries/reports/semantic_vs_statistical.sql
--
-- For every unordered pair of companies (A, B) we compute
--   return_corr = Pearson correlation of their daily returns (last year)
--   cosine_sim  = 1 - cosine distance of their description embeddings
-- and compare the AVERAGE return correlation of different groups of pairs:
--   semantic top-5 : B is one of A's 5 nearest description neighbours
--   (split into those in the same sector / a different sector, to see
--    whether embeddings add anything beyond the sector label)
--   same sector    : A and B share a yfinance sector
--   random sample  : pseudo-random pairs, same count as the semantic group
--   all pairs      : every pair, the baseline
-- Caveat: pairs share companies, so they are not independent samples;
-- treat differences as descriptive, not as a significance test.
-- ============================================================================
WITH last_year AS (
    SELECT cik, trade_date, close
    FROM prices
    WHERE trade_date > (SELECT max(trade_date) FROM prices) - INTERVAL '1 year'
),
daily_returns AS (
    -- daily return = close / previous close - 1, per company (see correlated_companies.sql)
    SELECT cik, trade_date,
           (close / LAG(close) OVER (PARTITION BY cik ORDER BY trade_date) - 1)::float8 AS ret
    FROM last_year
),
pair_corr AS (
    -- Return correlation for EVERY pair. a.cik < b.cik keeps each unordered
    -- pair once (A,B) and skips (B,A) and (A,A): 92*91/2 = 4186 pairs.
    SELECT a.cik AS cik_a,
           b.cik AS cik_b,
           corr(a.ret, b.ret) AS return_corr
    FROM daily_returns a
    JOIN daily_returns b
      ON b.trade_date = a.trade_date        -- align returns on the same day
     AND a.cik < b.cik
    GROUP BY a.cik, b.cik
),
pairs AS (
    -- Attach the semantic side (cosine similarity) and the sector flag.
    SELECT pc.cik_a,
           pc.cik_b,
           pc.return_corr,
           1 - (ca.description_embedding <=> cb.description_embedding) AS cosine_sim,
           ca.sector = cb.sector AS same_sector
    FROM pair_corr pc
    JOIN companies ca ON ca.cik = pc.cik_a
    JOIN companies cb ON cb.cik = pc.cik_b
),
semantic_pairs AS (
    -- Each company's 5 nearest description neighbours. LATERAL lets the
    -- subquery refer to the outer row c (a "for each company" loop in SQL).
    -- LEAST/GREATEST turn (A,B) and (B,A) into the same unordered pair, and
    -- DISTINCT removes pairs that are in each other's top-5 twice.
    SELECT DISTINCT LEAST(c.cik, n.cik)    AS cik_a,
                    GREATEST(c.cik, n.cik) AS cik_b
    FROM companies c
    CROSS JOIN LATERAL (
        SELECT o.cik
        FROM companies o
        WHERE o.cik <> c.cik
        ORDER BY o.description_embedding <=> c.description_embedding
        LIMIT 5
    ) AS n
),
random_pairs AS (
    -- Deterministic "random" sample: order pairs by a hash of their keys, so
    -- the report gives the same numbers on every run. Same size as the
    -- semantic group so the comparison is like-for-like.
    SELECT cik_a, cik_b
    FROM pairs
    ORDER BY md5(cik_a || cik_b)
    LIMIT (SELECT count(*) FROM semantic_pairs)
),
grouped AS (
    SELECT 1 AS sort_key, 'Semantic top-5 neighbours' AS pair_group, p.*
    FROM pairs p JOIN semantic_pairs USING (cik_a, cik_b)
    UNION ALL
    SELECT 2, '  ...of which same sector', p.*
    FROM pairs p JOIN semantic_pairs USING (cik_a, cik_b) WHERE p.same_sector
    UNION ALL
```

```
    SELECT 3, '  ...of which different sector', p.*
    FROM pairs p JOIN semantic_pairs USING (cik_a, cik_b) WHERE NOT p.same_sector
    UNION ALL
    SELECT 4, 'Same-sector pairs', p.* FROM pairs p WHERE p.same_sector
    UNION ALL
    SELECT 5, 'Random pairs (sample)', p.* FROM pairs p JOIN random_pairs USING (cik_a, cik_b)
    UNION ALL
    SELECT 6, 'All pairs (baseline)', p.* FROM pairs p
)
SELECT pair_group,
       count(*)                                  AS pairs,
       round(avg(return_corr)::numeric, 3)       AS avg_return_corr,
       round(stddev(return_corr)::numeric, 3)    AS sd_return_corr,
       round(avg(cosine_sim)::numeric, 3)        AS avg_cosine_sim
FROM grouped
GROUP BY sort_key, pair_group
ORDER BY sort_key;
```

db/queries/reports/semantic_vs_statistical_summary.sql: ROW_NUMBER window ranks, corr() with FILTER

```
-- ======================================================================
-- REPORT (companion): one-number answers to "does semantic similarity
-- predict return correlation?"
--   npx tsx scripts/run-sql.ts db/queries/reports/semantic_vs_statistical_summary.sql
--
--   corr_all_pairs     : Pearson corr between cosine_sim and return_corr
--                        across all 4186 pairs (does more similar text go
--                        with more similar price moves?)
--   corr_within_sector : the same, only for same-sector pairs; tests whether
--                        embeddings add information BEYOND the sector label
--   corr_cross_sector  : the same, only for different-sector pairs
--   avg_jaccard        : average over all companies of
--                        |semantic top-5 ∩ correlation top-5| / |union|
--   (0 = no overlap, 1 = identical lists; two random top-5 lists out of 91
--    companies would overlap by about 5*5/91 = 0.27 names, Jaccard ≈ 0.03)
-- ======================================================================
WITH last_year AS (
    SELECT cik, trade_date, close
    FROM prices
    WHERE trade_date > (SELECT max(trade_date) FROM prices) - INTERVAL '1 year'
),
daily_returns AS (
    SELECT cik, trade_date,
           (close / LAG(close) OVER (PARTITION BY cik ORDER BY trade_date) - 1)::float8 AS ret
    FROM last_year
),
directed_pairs AS (
    -- Both directions (A,B) and (B,A): needed to rank each company's
    -- neighbours from its own point of view.
    SELECT a.cik AS cik,
           b.cik AS other,
           corr(a.ret, b.ret) AS return_corr
    FROM daily_returns a
    JOIN daily_returns b
      ON b.trade_date = a.trade_date
     AND a.cik <> b.cik
    GROUP BY a.cik, b.cik
),
scored AS (
    SELECT d.cik,
           d.other,
           d.return_corr,
           1 - (ca.description_embedding <=> cb.description_embedding) AS cosine_sim,
           ca.sector = cb.sector AS same_sector,
           -- rank 1..91 of each company's neighbours, separately by each measure
           ROW_NUMBER() OVER (PARTITION BY d.cik ORDER BY d.return_corr DESC)
               AS corr_rank,
           ROW_NUMBER() OVER (PARTITION BY d.cik
                              ORDER BY ca.description_embedding <=> cb.description_embedding)
               AS semantic_rank
    FROM directed_pairs d
    JOIN companies ca ON ca.cik = d.cik
    JOIN companies cb ON cb.cik = d.other
),
jaccard_per_company AS (
    -- intersection = neighbours in BOTH top-5 lists; union = 5 + 5 - intersection
    SELECT cik,
           count(*) FILTER (WHERE corr_rank <= 5 AND semantic_rank <= 5) AS overlap
    FROM scored
    GROUP BY cik
```

```
)
SELECT
    -- each unordered pair appears twice in `scored`; that does not change a correlation
    round(corr(cosine_sim, return_correlation_all)::numeric, 3)                 AS
corr_all_pairs,
    round(corr(cosine_sim, return_correlation_all)
          FILTER (WHERE same_sector)::numeric, 3)                               AS
corr_within_sector,
    round(corr(cosine_sim, return_correlation_all)
          FILTER (WHERE NOT same_sector)::numeric, 3)                           AS
corr_cross_sector,
    (SELECT round(avg(overlap / (10.0 - overlap)), 3) FROM jaccard_per_company) AS avg_jaccard,
    (SELECT round(avg(overlap), 2) FROM jaccard_per_company)                    AS
avg_overlap_of_5
FROM (SELECT cosine_sim, return_corr AS return_correlation_all, same_sector FROM scored) s;
```

db/queries/reports/sector_embedding_distance.sql: self-join, AVG with FILTER, pgvector <=>

```
-- ======================================================================
-- REPORT (vector): Do the description embeddings capture sector structure?
-- For each sector: the average cosine DISTANCE (pgvector <=>) from its
-- companies to (a) other companies in the SAME sector and (b) companies in
-- OTHER sectors. If embeddings "know" sectors, within < across, and the
-- ratio across/within is well above 1.
-- Remember: the embeddings were built from description text ONLY; the
-- sector label was never shown to the model.
--   npx tsx scripts/run-sql.ts db/queries/reports/sector_embedding_distance.sql
-- ======================================================================
SELECT a.sector,
       count(DISTINCT a.cik) AS companies,
       round(avg(a.description_embedding <=> b.description_embedding)
             FILTER (WHERE b.sector = a.sector)::numeric, 3)   AS within_sector_distance,
       round(avg(a.description_embedding <=> b.description_embedding)
             FILTER (WHERE b.sector <> a.sector)::numeric, 3)  AS cross_sector_distance,
       round((avg(a.description_embedding <=> b.description_embedding)
                  FILTER (WHERE b.sector <> a.sector)
            / avg(a.description_embedding <=> b.description_embedding)
                  FILTER (WHERE b.sector = a.sector))::numeric, 2) AS ratio_cross_to_within
FROM companies a
JOIN companies b              -- every ordered pair of DIFFERENT companies
  ON b.cik <> a.cik
GROUP BY a.sector
ORDER BY ratio_cross_to_within DESC;
```

7. NoSQL/ Vector Database Commands

What is embedded: each company's business description (text only; sector and industry are deliberately left out so the sector analysis is not circular). **Model:** all-MiniLM-L6-v2 from sentence-transformers, 384 dimensions, `normalize_embeddings=True` so every vector has length 1. **Storage:** column `companies.description_embedding VECTOR(384)` on the company's own row. **Metric:** cosine distance, operator `<=>` (0 = same direction ... 2 = opposite; similarity = 1 - distance). **Index:** HNSW with `vector_cosine_ops`. **Retrieval:** top-k nearest neighbours of a company with `ORDER BY ... <=> ... LIMIT 5`.

a) Enable pgvector and create the vector column

```
CREATE EXTENSION IF NOT EXISTS vector;          -- db/schema.sql
...
description_embedding VECTOR(384),             -- all-MiniLM-L6-v2, L2-normalised (companies
table)
```

```
equivector=> SELECT extname, extversion FROM pg_extension WHERE extname = 'vector';
 extname | extversion
---------+------------
 vector  | 0.8.2
(1 row)
```

Output 7.1: pgvector installed on the live Supabase database

```
equivector=> SELECT column_name, format_type(atttypid, atttypmod) AS type FROM
equivector-> information_schema.columns c JOIN pg_attribute a ON a.attrelid = 'companies'::regclass
equivector-> AND a.attname = c.column_name WHERE c.table_name = 'companies' AND c.column_name =
equivector-> 'description_embedding';
      column_name      |    type
-----------------------+-------------
 description_embedding | vector(384)
(1 row)
```

Output 7.2: the vector column in the live database

b) Create the embeddings (seed/embed_descriptions.py)

```
    with psycopg.connect(url) as conn, conn.cursor() as cur:
        cur.execute(
            "SELECT cik, description FROM companies "
            "WHERE description IS NOT NULL "
            + ("" if args.all else "AND description_embedding IS NULL ")
            + "ORDER BY cik"
        )
        rows = cur.fetchall()
        if not rows:
            print("nothing to embed")
            return

        model = SentenceTransformer(MODEL)
        print(f"model {MODEL}: dim={model.get_embedding_dimension()}, "
              f"max_seq_length={model.max_seq_length}")
        vectors = model.encode([d for _, d in rows], normalize_embeddings=True,
                               batch_size=32, show_progress_bar=False)

        # One transaction for all updates (committed when the `with` block exits).
        cur.executemany(
            "UPDATE companies SET description_embedding = %s::vector WHERE cik = %s",
            [(to_pgvector(v), cik) for (cik, _), v in zip(rows, vectors)],
        )
```

```
        print(f"embedded {len(rows)} companies")

        # Sanity check inside the DB: dimension and unit length of what was stored.
        cur.execute("""
            SELECT count(*), min(vector_dims(description_embedding)),
                   round(min(vector_norm(description_embedding))::numeric, 4),
                   round(max(vector_norm(description_embedding))::numeric, 4)
            FROM companies WHERE description_embedding IS NOT NULL""")
        n, dims, nmin, nmax = cur.fetchone()
        print(f"stored: {n} vectors, dims={dims}, norm range=[{nmin}, {nmax}]")
```

```
equivector=> SELECT count(*) AS vectors, min(vector_dims(description_embedding)) AS dims,
equivector-> round(min(vector_norm(description_embedding))::numeric, 4) AS min_norm,
equivector-> round(max(vector_norm(description_embedding))::numeric, 4) AS max_norm FROM companies
equivector-> WHERE description_embedding IS NOT NULL;
 vectors | dims | min_norm | max_norm
---------+------+----------+----------
      92 |  384 |   1.0000 |   1.0000
(1 row)
```

Output 7.3: 92 stored vectors, 384 dimensions, all of length 1

```
equivector=> SELECT ticker, left(description_embedding::text, 60) || '...' AS first_values FROM
equivector-> companies WHERE ticker IN ('JPM','MS') ORDER BY ticker;
 ticker |                          first_values
--------+---------------------------------------------------------------
 JPM    | [0.0294668,-0.0538808,-0.0488283,-1.11e-05,0.0191394,-0.0379...
 MS     | [0.0117391,-0.0537996,-0.0584434,0.0284171,-0.0076174,0.0235...
(2 rows)
```

Output 7.4: the first values of two stored vectors

c) Create the HNSW index

```
-- 2. HNSW index on the description embeddings (pgvector).
--    HNSW = Hierarchical Navigable Small World: a layered graph in which
--    each vector links to its near neighbours, so a nearest-neighbour search
--    hops through the graph instead of comparing against every row.
--    It is approximate (ANN): very fast, may very occasionally miss a true
--    neighbour.
--    The operator class MUST match the operator used in queries:
--      vector_cosine_ops  <->  ORDER BY description_embedding <=> $query
--    (a vector_l2_ops index would be ignored by a <=> query).
--    m / ef_construction are pgvector's defaults, written out for clarity:
--      m = 16               links per node per layer
--      ef_construction = 64 candidate list size while building
--    With 92 rows the planner may still choose a sequential scan (cheaper at
--    this size); the index is what keeps the search fast as the table grows.
CREATE INDEX IF NOT EXISTS idx_companies_embedding
    ON companies USING hnsw (description_embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);
```

```
equivector=> \x on
equivector=> SELECT indexname, indexdef FROM pg_indexes WHERE indexname = 'idx_companies_embedding';
-[ RECORD 1
]-----------------------------------------------------------------------------------------------------------------------
--------------------------------------
indexname | idx_companies_embedding
indexdef  | CREATE INDEX idx_companies_embedding ON public.companies USING hnsw (description_embedding
vector_cosine_ops) WITH (m='16', ef_construction='64')
```

Output 7.5: the index as stored in the live database (psql expanded output)

d) Similarity query (read) and its real output

```
-- ============================================================================
-- SEMANTIC similarity: top-5 nearest neighbours of one company by
-- description embedding (pgvector).
--   $1 = ticker, e.g. 'JPM'
--   npx tsx scripts/run-sql.ts db/queries/similar_companies.sql JPM
```

```
--
-- <=> is pgvector's COSINE DISTANCE operator:
--   0 = same direction (same meaning) ... 2 = opposite.
--   similarity = 1 - distance.
--
-- Why a scalar subquery and not a JOIN to the target row:
-- the HNSW index (vector_cosine_ops) can only be used when the ORDER BY is
--   <indexed column> <=> <one fixed query vector>.
-- The uncorrelated subquery below is evaluated ONCE (an "InitPlan") and acts
-- as that fixed vector, so EXPLAIN shows "Index Scan using
-- idx_companies_embedding". Written as a join (FROM companies c, target t),
-- the planner must compute the distance for every row and sort, and the
-- index is never used (verified with EXPLAIN, see docs/evidence/).
-- ======================================================================
SELECT ticker,
       name,
       sector,
       industry,
       round((1 - (description_embedding <=> q.emb))::numeric, 3) AS cosine_similarity
FROM companies,
     -- the query vector, fetched once; reused in SELECT for the similarity
     (SELECT description_embedding AS emb FROM companies WHERE ticker = $1) AS q
WHERE ticker <> $1                                  -- exclude the company itself
ORDER BY description_embedding <=> (SELECT description_embedding
                                    FROM companies
                                    WHERE ticker = $1)  -- nearest first
LIMIT 5;
```

```
equivector=> -- db/queries/similar_companies.sql ($1 = JPM)
 ticker |            name              |      sector       |      industry       | cosine_similarity
--------+------------------------------+-------------------+---------------------+-------------------
 MS     | Morgan Stanley               | Financial Services | Capital Markets     |             0.635
 GS     | The Goldman Sachs Group, Inc. | Financial Services | Capital Markets     |             0.615
 BAC    | Bank of America Corporation  | Financial Services | Banks - Diversified |             0.589
 WFC    | Wells Fargo & Company        | Financial Services | Banks - Diversified |             0.526
 MA     | Mastercard Incorporated      | Financial Services | Credit Services     |             0.460
(5 rows)
```

Output 7.6: JPMorgan's 5 nearest description neighbours (live)

```
equivector=> EXPLAIN (COSTS OFF) SELECT ticker FROM companies WHERE ticker <> 'JPM' ORDER BY
equivector-> description_embedding <=> (SELECT description_embedding FROM companies WHERE ticker =
equivector-> 'JPM') LIMIT 5;
                            QUERY PLAN
------------------------------------------------------------------------
 Limit
   InitPlan 1
     ->  Index Scan using companies_ticker_key on companies companies_1
           Index Cond: ((ticker)::text = 'JPM'::text)
   ->  Index Scan using idx_companies_embedding on companies
         Order By: (description_embedding <=> (InitPlan 1).col1)
         Filter: ((ticker)::text <> 'JPM'::text)
(7 rows)
```

Output 7.7: EXPLAIN shows the HNSW index (idx_companies_embedding) is used

e) Vector analysis queries

```
equivector=> -- db/queries/reports/sector_embedding_distance.sql
         sector         | companies | within_sector_distance | cross_sector_distance | ratio_cross_to_within
------------------------+-----------+------------------------+-----------------------+-----------------------
 Utilities              |         6 |                  0.528 |                 0.797 |                  1.51
 Healthcare             |         9 |                  0.573 |                 0.820 |                  1.43
 Consumer Defensive     |         8 |                  0.556 |                 0.769 |                  1.38
 Financial Services     |        10 |                  0.567 |                 0.775 |                  1.37
 Energy                 |         7 |                  0.586 |                 0.787 |                  1.34
 Technology             |        12 |                  0.617 |                 0.786 |                  1.27
 Communication Services |         8 |                  0.644 |                 0.789 |                  1.22
 Industrials            |         9 |                  0.675 |                 0.782 |                  1.16
 Consumer Cyclical      |        10 |                  0.684 |                 0.776 |                  1.14
 Real Estate            |         6 |                  0.755 |                 0.834 |                  1.10
 Basic Materials        |         7 |                  0.740 |                 0.801 |                  1.08
(11 rows)
```

Output 7.8: in all 11 sectors, descriptions are closer within the sector than across sectors (embeddings never saw the sector label)

```
equivector=> -- db/queries/reports/semantic_vs_statistical_summary.sql
 corr_all_pairs | corr_within_sector | corr_cross_sector | avg_jaccard | avg_overlap_of_5
----------------+--------------------+-------------------+-------------+------------------
          0.280 |              0.536 |             0.076 |       0.291 |             1.99
(1 row)
```

Output 7.9: correlation between cosine similarity and return correlation over all pairs, within and across sectors

Create/update/delete of vectors: vectors are created and updated by `seed/embed_descriptions.py` (UPDATE shown above); `seed/load_data.py` sets an embedding back to NULL when a description changes; a vector is deleted together with its company row. The web app itself only reads vectors (similarity search); it has no screen that edits or deletes them.

8. Results

a) Resulting tables (real psql output from the live Supabase database, 2026-11-09)

Each figure below is the text output of `psql` against the live database, rendered as an image. Nothing was typed by hand. **Data note:** the `prices` table contains one hand-inserted test row (AAPL, 2026-10-30, close 255, volume 1,000,000; audit row 484, logged 2026-10-31 05:42 UTC) that is not from the data pipeline. Because it is the only row on that date, the `top_movers` view currently compares AAPL alone (Output 8.7), and AAPL/Technology figures include it. **[PLACEHOLDER: delete this test row and re-run, or keep this note]**

```
equivector=> SELECT 'companies' AS table_name, count(*) FROM companies UNION ALL SELECT 'prices',
equivector-> count(*) FROM prices UNION ALL SELECT 'profiles', count(*) FROM profiles UNION ALL
equivector-> SELECT 'watchlists', count(*) FROM watchlists UNION ALL SELECT 'activity_log',
equivector-> count(*) FROM activity_log;
  table_name  | count
--------------+-------
 companies    |    92
 prices       | 25301
 profiles     |     3
 watchlists   |     4
 activity_log |   478
(5 rows)
```

Output 8.1: row counts of the five tables

```
equivector=> SELECT cik, ticker, name, sector, industry, last_updated FROM companies ORDER BY
equivector-> ticker LIMIT 6;
    cik     | ticker |                name                |   sector   |           industry            |         last_updated
------------+--------+------------------------------------+------------+-------------------------------+-------------------------------
 0000320193 | AAPL   | Apple Inc.                         | Technology | Consumer Electronics          | 2026-10-31 05:42:06.350239+00
 0001551152 | ABBV   | AbbVie Inc.                        | Healthcare | Drug Manufacturers - General  | 2026-10-25 14:29:36.858802+00
 0000001800 | ABT    | Abbott Laboratories                | Healthcare | Medical Devices               | 2026-10-25 14:29:36.858802+00
 0000796343 | ADBE   | Adobe Inc.                         | Technology | Software - Application        | 2026-10-25 14:29:36.858802+00
 0000004904 | AEP    | American Electric Power Company, Inc. | Utilities  | Utilities - Regulated Electric | 2026-10-25 14:29:36.858802+00
 0000002488 | AMD    | Advanced Micro Devices, Inc.       | Technology | Semiconductors                | 2026-10-25 14:29:36.858802+00
(6 rows)
```

Output 8.2: companies (first 6 by ticker)

```
equivector=> SELECT * FROM prices WHERE cik = '0000019617' ORDER BY trade_date DESC LIMIT 5;
    cik     | trade_date |   open   |   high   |   low    |  close   | volume
------------+------------+----------+----------+----------+----------+---------
 0000019617 | 2026-10-23 | 333.0000 | 335.1400 | 330.4000 | 331.5800 | 4779500
 0000019617 | 2026-10-22 | 332.2800 | 336.6200 | 332.2600 | 336.5400 | 6199100
 0000019617 | 2026-10-21 | 337.1200 | 337.1200 | 332.6500 | 335.1900 | 5770600
 0000019617 | 2026-10-20 | 333.9700 | 339.8600 | 332.5500 | 335.2600 | 6157500
 0000019617 | 2026-10-19 | 339.2200 | 341.0100 | 333.2200 | 333.5300 | 6925800
(5 rows)
```

Output 8.3: prices for JPMorgan (cik 0000019617), latest 5 days

```
equivector=> SELECT display_name, role, created_at FROM profiles ORDER BY created_at;
 display_name  |  role   |          created_at
---------------+---------+-------------------------------
 Demo Analyst  | analyst | 2026-10-25 14:43:00.437916+00
 riyanakarthik | viewer  | 2026-10-30 12:38:06.338282+00
 analyst       | analyst | 2026-10-30 13:44:03.277632+00
(3 rows)
```

Output 8.4: profiles with their roles

```
equivector=> SELECT w.user_id, c.ticker, w.note, w.created_at, w.updated_at FROM watchlists w JOIN
equivector-> companies c USING (cik) ORDER BY w.created_at;
              user_id               | ticker |          note           |          created_at           |          updated_at
------------------------------------+--------+-------------------------+-------------------------------+-------------------------------
 f711fd0b-2c14-4e17-8c06-a25c2c435fea | NVDA   | AI chips bellwether     | 2026-10-25 14:49:12.947408+00 |
 f711fd0b-2c14-4e17-8c06-a25c2c435fea | JPM    | Largest US bank         | 2026-10-25 14:49:13.742284+00 |
 f711fd0b-2c14-4e17-8c06-a25c2c435fea | XOM    | Watch oil price exposure | 2026-10-25 14:49:14.69237+00  | 2026-10-25 14:49:18.382237+00
 935177c8-677a-4f88-8037-60df34fbc29e | MSFT   | role test: note edited  | 2026-10-30 13:52:52.757649+00 | 2026-10-30 13:52:54.692663+00
(4 rows)
```

Output 8.5: watchlists (the M:N relation) joined to companies

```
equivector=> SELECT log_id, action, cik, user_id, logged_at FROM activity_log ORDER BY log_id DESC
equivector-> LIMIT 6;
 log_id |      action      |    cik     |               user_id                |           logged_at
--------+------------------+------------+--------------------------------------+-------------------------------
    484 | PRICE_INSERT     | 0000320193 |                                      | 2026-10-31 05:42:06.350239+00
    483 | WATCHLIST_UPDATE | 0000789019 | 935177c8-677a-4f88-8037-60df34fbc29e | 2026-10-30 13:52:54.692663+00
    482 | WATCHLIST_ADD    | 0000789019 | 935177c8-677a-4f88-8037-60df34fbc29e | 2026-10-30 13:52:52.757649+00
    481 | WATCHLIST_REMOVE | 0000789019 | 935177c8-677a-4f88-8037-60df34fbc29e | 2026-10-30 13:52:51.817121+00
    480 | WATCHLIST_UPDATE | 0000789019 | 935177c8-677a-4f88-8037-60df34fbc29e | 2026-10-30 13:45:28.719426+00
    479 | WATCHLIST_ADD    | 0000789019 | 935177c8-677a-4f88-8037-60df34fbc29e | 2026-10-30 13:45:26.634865+00
(6 rows)
```

Output 8.6: activity_log, newest rows (PRICE_INSERT from the trigger, WATCHLIST_* from transactions)

```
equivector=> SELECT ticker, sector, prev_date, trade_date, prev_close, close, pct_change FROM
equivector-> top_movers;
 ticker |   sector    | prev_date  | trade_date | prev_close |  close   | pct_change
--------+-------------+------------+------------+------------+----------+------------
 AAPL   | Technology | 2026-10-23 | 2026-10-30 |   347.7300 | 255.0000 |     -26.67
(1 row)
```

Output 8.7: the top_movers view (affected by the AAPL test row, see the note above)

```
equivector=> -- db/queries/reports/sector_having.sql ($1 = 6, $2 = 0)
       sector       | companies | avg_pct_change_1y | worst_company_pct | best_company_pct
--------------------+-----------+-------------------+-------------------+------------------
 Energy             |         7 |             68.04 |             50.74 |           114.17
 Healthcare         |         9 |             34.74 |            -14.03 |            73.92
 Basic Materials    |         7 |             28.95 |             -1.36 |            57.76
 Technology         |        12 |             21.46 |            -51.32 |           117.45
 Consumer Defensive |         8 |             15.27 |             -3.30 |            38.13
 Industrials        |         9 |             15.19 |             -7.05 |            41.57
 Financial Services |        10 |              3.73 |            -16.77 |            20.67
(7 rows)
```

Output 8.8: sector_having.sql with $1 = 6 and $2 = 0 (GROUP BY + HAVING)

```
equivector=> -- db/queries/reports/above_own_average.sql
 ticker |                 name                 |         sector         | latest_close | avg_close | pct_above_avg
--------+--------------------------------------+------------------------+--------------+-----------+---------------
 PSX    | Phillips 66                          | Energy                 |     283.1800 |    177.01 |         59.98
 AMD    | Advanced Micro Devices, Inc.         | Technology             |     516.7900 |    351.95 |         46.84
 COP    | ConocoPhillips                       | Energy                 |     150.8000 |    110.74 |         36.18
 CVX    | Chevron Corporation                  | Energy                 |     233.2100 |    178.15 |         30.91
 XOM    | ExxonMobil Holdings Corporation      | Energy                 |     183.2100 |    141.38 |         29.59
 EOG    | EOG Resources, Inc.                  | Energy                 |     163.0300 |    125.81 |         29.58
 MRK    | Merck & Co., Inc.                    | Healthcare             |     145.2700 |    115.40 |         25.89
 AMGN   | Amgen Inc.                           | Healthcare             |     439.2600 |    351.70 |         24.90
 OXY    | Occidental Petroleum Corporation     | Energy                 |      64.1700 |     51.44 |         24.76
 ABBV   | AbbVie Inc.                          | Healthcare             |     285.6400 |    229.10 |         24.68
 TXN    | Texas Instruments Incorporated       | Technology             |     284.6900 |    231.94 |         22.75
 TMO    | Thermo Fisher Scientific Inc.        | Healthcare             |     673.8200 |    550.85 |         22.32
... (first 12 rows shown)
(59 rows)
```

Output 8.9: above_own_average.sql (correlated subqueries)

```
equivector=> -- db/queries/reports/most_watched.sql
 ticker |              name               |       sector       | watchers |          last_added
--------+---------------------------------+--------------------+----------+-------------------------------
 JPM    | JPMorgan Chase & Co.            | Financial Services |        1 | 2026-10-25 14:49:13.742284+00
 MSFT   | Microsoft Corporation           | Technology         |        1 | 2026-10-30 13:52:52.757649+00
 NVDA   | NVIDIA Corporation              | Technology         |        1 | 2026-10-25 14:49:12.947408+00
 XOM    | ExxonMobil Holdings Corporation | Energy             |        1 | 2026-10-25 14:49:14.69237+00
(4 rows)
```

Output 8.10: most_watched.sql (JOIN on the M:N table)

```
equivector=> -- db/queries/reports/activity_summary.sql
      action      | events | companies | users |           first_at            |            last_at
------------------+--------+-----------+-------+-------------------------------+-------------------------------
 PRICE_INSERT     |    467 |        92 |     0 | 2026-10-25 14:29:36.858802+00 | 2026-10-31 05:42:06.350239+00
 WATCHLIST_ADD    |      6 |         5 |     2 | 2026-10-25 14:49:12.947408+00 | 2026-10-30 13:52:52.757649+00
 WATCHLIST_UPDATE |      3 |         2 |     2 | 2026-10-25 14:49:18.382237+00 | 2026-10-30 13:52:54.692663+00
 WATCHLIST_REMOVE |      2 |         2 |     2 | 2026-10-25 14:49:20.942261+00 | 2026-10-30 13:52:51.817121+00
(4 rows)
```

Output 8.11: activity_summary.sql (GROUP BY, COUNT DISTINCT)

```
equivector=> -- db/queries/correlated_companies.sql ($1 = JPM)
 ticker |            name              |       sector       |      industry       | return_correlation | days_used
--------+------------------------------+--------------------+---------------------+--------------------+-----------
 BAC    | Bank of America Corporation  | Financial Services | Banks - Diversified |              0.742 |       245
 WFC    | Wells Fargo & Company        | Financial Services | Banks - Diversified |              0.718 |       245
 MS     | Morgan Stanley               | Financial Services | Capital Markets     |              0.621 |       245
 GS     | The Goldman Sachs Group, Inc. | Financial Services | Capital Markets     |              0.608 |       245
 AXP    | American Express Company     | Financial Services | Credit Services     |              0.552 |       245
(5 rows)
```

Output 8.12: correlated_companies.sql for JPM (CTE + LAG + corr())

```
equivector=> -- db/queries/reports/semantic_vs_statistical.sql
           pair_group           | pairs | avg_return_corr | sd_return_corr | avg_cosine_sim
--------------------------------+-------+-----------------+----------------+----------------
 Semantic top-5 neighbours      |   307 |           0.297 |          0.271 |          0.445
   ...of which same sector      |   154 |           0.464 |          0.228 |          0.485
   ...of which different sector |   153 |           0.130 |          0.197 |          0.406
 Same-sector pairs              |   356 |           0.324 |          0.237 |          0.375
 Random pairs (sample)          |   307 |           0.095 |          0.190 |          0.217
 All pairs (baseline)           |  4186 |           0.103 |          0.188 |          0.223
(6 rows)
```

Output 8.13: semantic_vs_statistical.sql: average return correlation by kind of pair

Finding (Output 8.13): description neighbours co-move about three times more than random pairs (0.297 vs 0.095) but less than plain same-sector pairs (0.324). Within a sector they co-move more than average (0.464); across sectors barely more than random (0.130). Semantic similarity predicts co-movement mostly through the sector, not beyond it. These live values differ slightly from those recorded in `docs/problem-statement.md` (0.295 / 0.321 / 0.461 / 0.129), which were measured on earlier data.

9. Next.js-Project Tree Structure

Real output of `tree -a --dirsfirst -I '.git|node_modules|.next|.venv|__pycache__|.DS_Store'` in the project root (2026-11-09). `.env.local` and `seed/data/raw/` are git-ignored; `.env.local` holds the secrets and is listed by name only.

```
.
├── app
│   ├── api
│   │   ├── companies
│   │   │   └── route.ts
│   │   ├── health
│   │   │   └── route.ts
│   │   ├── me
│   │   │   └── route.ts
│   │   ├── prices
│   │   │   └── route.ts
│   │   ├── reports
│   │   │   └── [slug]
│   │   │       └── route.ts
│   │   ├── similar
│   │   │   └── route.ts
│   │   ├── top-movers
│   │   │   └── route.ts
│   │   └── watchlist
│   │       └── route.ts
│   ├── auth
│   │   └── callback
│   │       └── route.ts
│   ├── dashboard
│   │   └── page.tsx
│   ├── login
│   │   └── page.tsx
│   ├── reports
│   │   └── page.tsx
│   ├── search
│   │   └── page.tsx
│   ├── signup
│   │   └── page.tsx
│   ├── watchlist
│   │   └── page.tsx
│   ├── favicon.ico
│   ├── globals.css
│   ├── layout.tsx
│   └── page.tsx
├── components
│   ├── ui
│   │   ├── badge.tsx
│   │   ├── button.tsx
│   │   ├── card.tsx
│   │   ├── input.tsx
│   │   └── table.tsx
│   ├── auth-form.tsx
│   ├── charts.tsx
│   ├── data-table.tsx
│   └── nav.tsx
├── db
│   ├── admin
│   │   └── set_role.sql
│   ├── demo
│   │   └── fire_trigger_live.sql
│   ├── local
│   │   └── auth_stub.sql
│   ├── migrations
│   │   ├── 001_watchlists_add_updated_at.sql
│   │   └── 002_drop_redundant_index.sql
│   ├── queries
│   │   ├── auth
│   │   │   └── profile_by_user.sql
│   │   ├── reports
│   │   │   ├── above_own_average.sql
│   │   │   ├── activity_summary.sql
│   │   │   ├── most_watched.sql
│   │   │   ├── sector_embedding_distance.sql
│   │   │   ├── sector_having.sql
│   │   │   ├── sector_summary.sql
│   │   │   ├── semantic_vs_statistical.sql
│   │   │   ├── semantic_vs_statistical_summary.sql
│   │   │   └── top_movers_top10.sql
```

```
│   │   ├── watchlist
│   │   │   ├── delete.sql
│   │   │   ├── insert.sql
│   │   │   ├── list.sql
│   │   │   ├── log_action.sql
│   │   │   └── update_note.sql
│   │   ├── companies_search.sql
│   │   ├── company_by_ticker.sql
│   │   ├── correlated_companies.sql
│   │   ├── prices_for_company.sql
│   │   ├── similar_companies.sql
│   │   └── top_movers.sql
│   ├── tests
│   │   ├── constraints_test.sql
│   │   ├── role_test.sql
│   │   └── trigger_snapshot.sql
│   ├── auth_profiles.sql
│   ├── indexes.sql
│   ├── role_enforcement.sql
│   ├── schema.sql
│   ├── security.sql
│   ├── triggers.sql
│   └── views.sql
├── docs
│   ├── evidence
│   │   ├── phase2_load_batch2.txt
│   │   ├── phase2_migrations.txt
│   │   ├── phase2_query_plans.txt
│   │   ├── phase2_semantic_neighbours.txt
│   │   ├── phase2_trigger_after.txt
│   │   ├── phase2_trigger_before.txt
│   │   ├── phase3_live_trigger_demo.txt
│   │   ├── phase3_rollback_proof.txt
│   │   ├── phase3_semantic_vs_statistical.txt
│   │   ├── phase4_auth_profiles.txt
│   │   ├── phase4_role_test.txt
│   │   ├── phase4_rollback_proof.txt
│   │   ├── phase5_T1_constraints.txt
│   │   ├── phase5_T2_trigger.txt
│   │   ├── phase5_T3_rollback.txt
│   │   ├── phase5_T4_role_db.txt
│   │   ├── phase5_T5_http_roles.txt
│   │   └── phase5_T6_signup_trigger.txt
│   ├── screenshots
│   │   ├── dashboard.png
│   │   ├── login.png
│   │   ├── reports.png
│   │   ├── search.png
│   │   └── watchlist.png
│   ├── DBMS-Project-Template.docx
│   ├── ER-diagram.png
│   ├── architecture.md
│   ├── conclusion-and-future-work.md
│   ├── data-dictionary.md
│   ├── database-choice-and-justification.md
│   ├── demo-script.md
│   ├── er-diagram.dot
│   ├── functional-requirements.md
│   ├── important-queries.md
│   ├── objectives.md
│   ├── problem-statement.md
│   ├── rubric-checklist.md
│   ├── scope.md
│   ├── testing-results.md
│   └── vector-implementation.md
├── lib
│   ├── supabase
│   │   ├── client.ts
│   │   ├── config.ts
│   │   └── server.ts
│   ├── api.ts
│   ├── auth.ts
│   ├── db.ts
│   ├── reports.ts
│   ├── sql.ts
│   ├── use-fetch.ts
│   ├── use-me.ts
│   ├── utils.ts
│   └── watchlist.ts
├── public
├── scripts
│   ├── prove-rollback.ts
│   ├── run-sql.ts
│   └── test-roles.ts
├── seed
│   ├── data
```

```
│   │   ├── clean
│   │   │   ├── clean_report.json
│   │   │   ├── companies.csv
│   │   │   ├── prices_batch1.csv
│   │   │   └── prices_batch2.csv
│   │   └── raw
│   │       ├── fetch_report.json
│   │       ├── info.csv
│   │       ├── prices.csv
│   │       └── sec_company_tickers.json
│   ├── clean_data.py
│   ├── embed_descriptions.py
│   ├── fetch_data.py
│   ├── load_data.py
│   ├── requirements.txt
│   └── tickers.txt
├── .env.local
├── .env.local.example
├── .gitignore
├── ARCHITECTURE.md
├── CLAUDE.md
├── DBMS Level 3 Project Requirements.pdf
├── README.md
├── components.json
├── eslint.config.mjs
├── middleware.ts
├── next-env.d.ts
├── next.config.ts
├── package-lock.json
├── package.json
├── postcss.config.mjs
├── tsconfig.json
└── tsconfig.tsbuildinfo

43 directories, 149 files
```

10. Few sample Screenshots of application code and demo

Captured on 2026-11-09 with Playwright (headless Chromium) from the app running locally (`npm run dev`) against the live Supabase database, logged in with the team's real analyst and viewer test accounts. No data was changed while capturing.

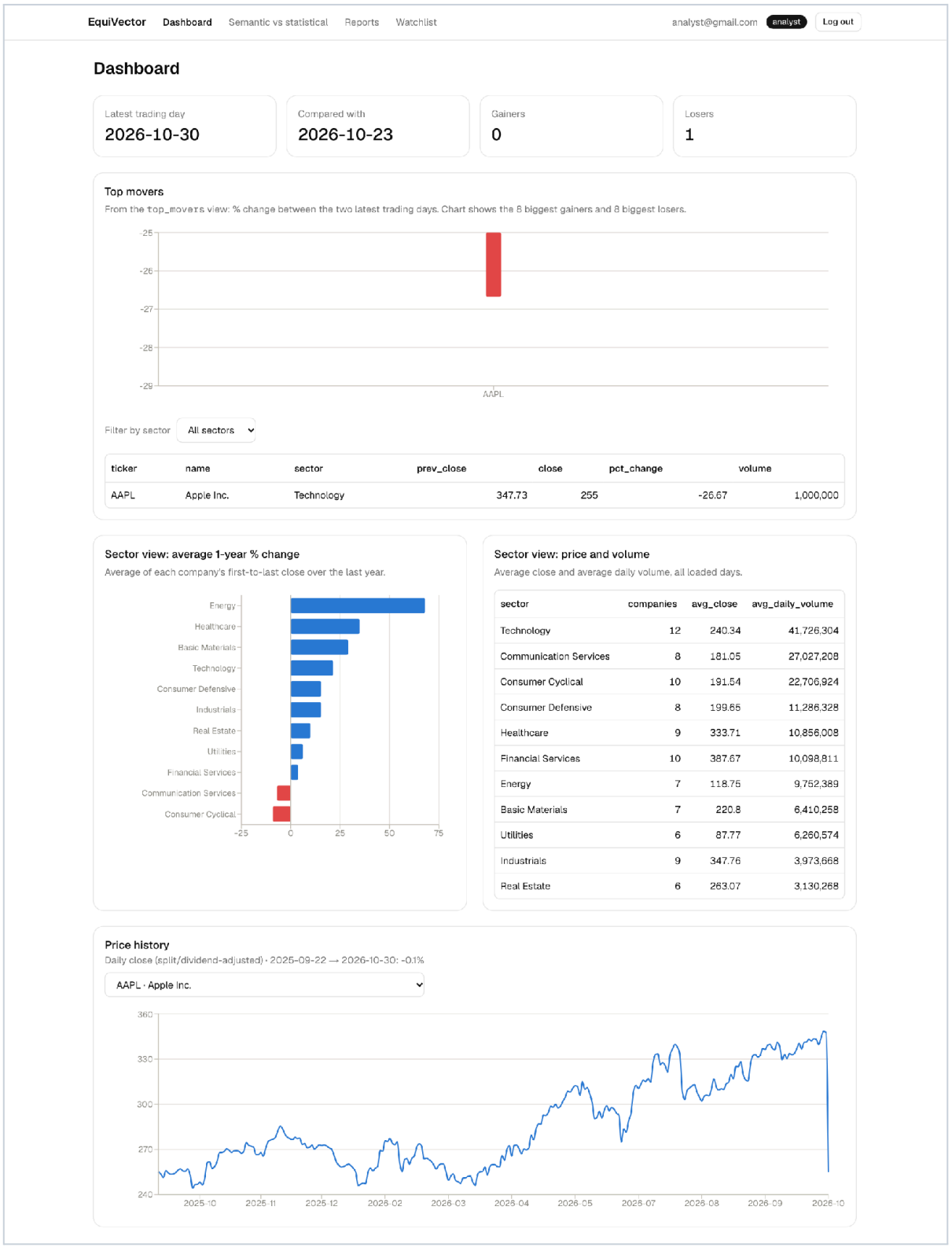

EquiVector
Dashboard
Semantic vs statistical
Reports
Watchlist
analyst@gmail.com
analyst
Log out
Dashboard
Latest trading day
2026-10-30
Compared with
2026-10-23
Gainers
0
Losers
1
Top movers
From the top_movers view: % change between the two latest trading days. Chart shows the 8 biggest gainers and 8 biggest losers.
-25
-26
-27
-28
-29
AAPL
Filter by sector
All sectors
ticker name sector prev_close close pct_change volume
AAPL Apple Inc. Technology 347.73 255 -26.67 1,000,000
Sector view: average 1-year % change
Average of each company's first-to-last close over the last year.
Energy
Healthcare
Basic Materials
Technology
Consumer Defensive
Industrials
Real Estate
Utilities
Financial Services
Communication Services
Consumer Cyclical
-25
0
25
50
75
Sector view: price and volume
Average close and average daily volume, all loaded days.
sector companies avg_close avg_daily_volume
Technology 12 240.34 41,726,304
Communication Services 8 181.05 27,027,208
Consumer Cyclical 10 191.54 22,706,924
Consumer Defensive 8 199.65 11,286,328
Healthcare 9 333.71 10,856,008
Financial Services 10 387.67 10,098,811
Energy 7 118.75 9,752,389
Basic Materials 7 220.8 6,410,258
Utilities 6 87.77 6,260,574
Industrials 9 347.76 3,973,668
Real Estate 6 263.07 3,130,268
Price history
Daily close (split/dividend-adjusted) · 2025-09-22 → 2026-10-30: -0.1%
AAPL · Apple Inc.
360
330
300
270
240
2025-10
2025-11
2025-12
2026-02
2026-03
2026-04
2026-05
2026-07
2026-08
2026-09
2026-10

Figure 10.1: Dashboard as analyst. The Top movers card shows only AAPL because of the test row described in Section 8; the price-history drop at the right edge is the same row.

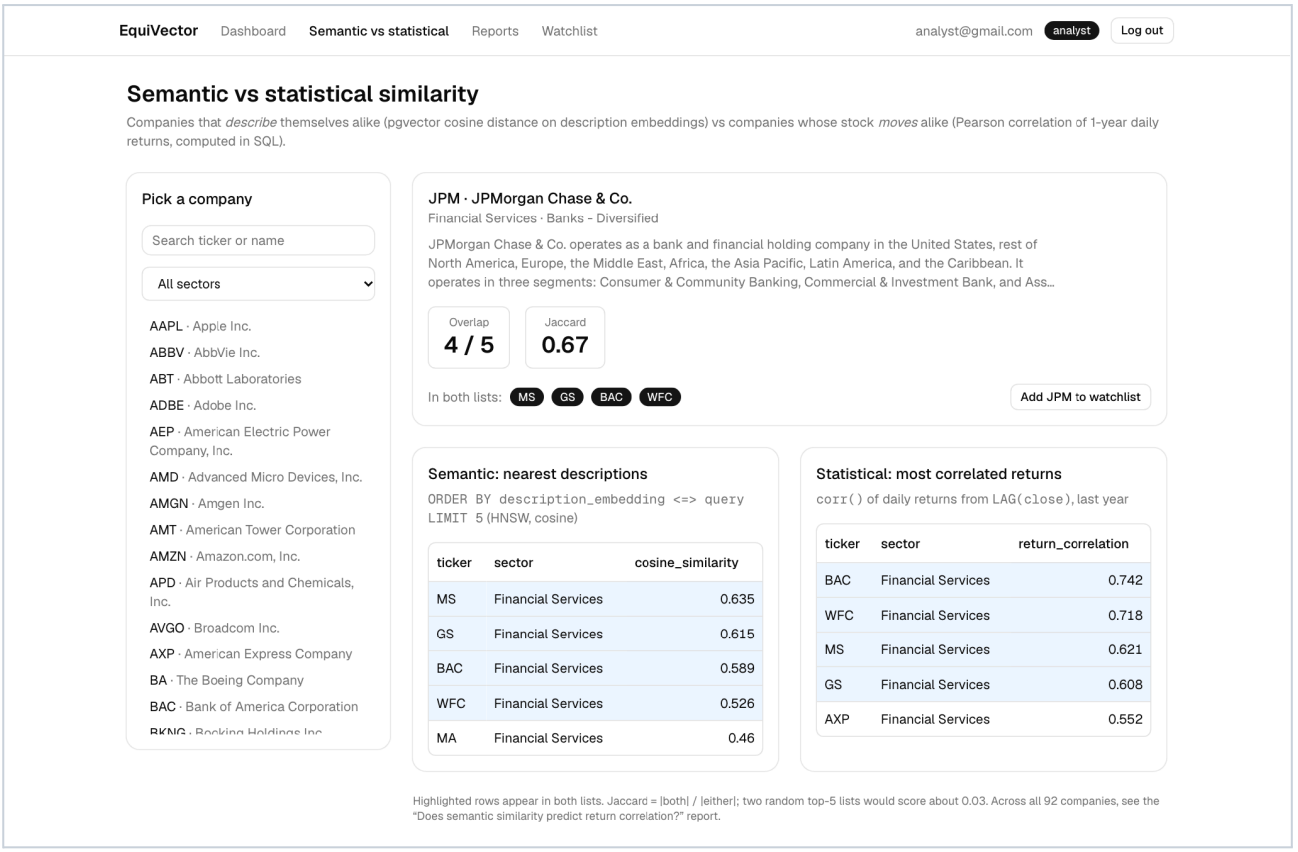
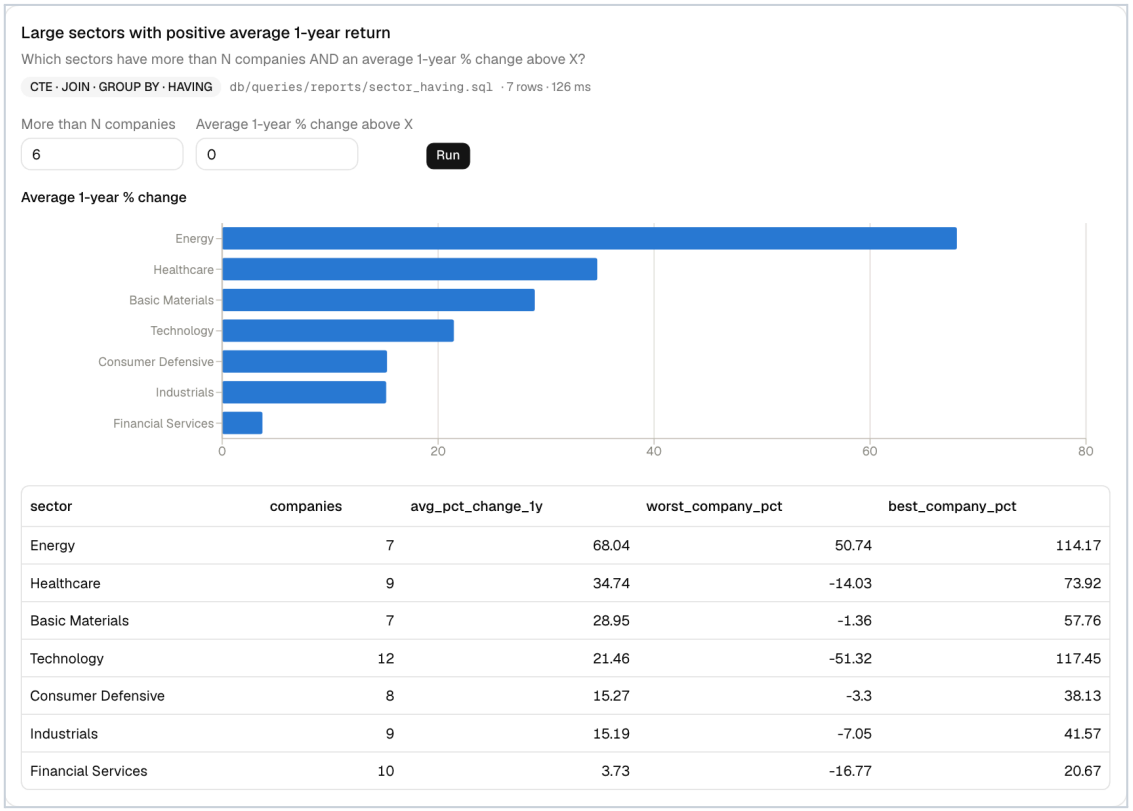
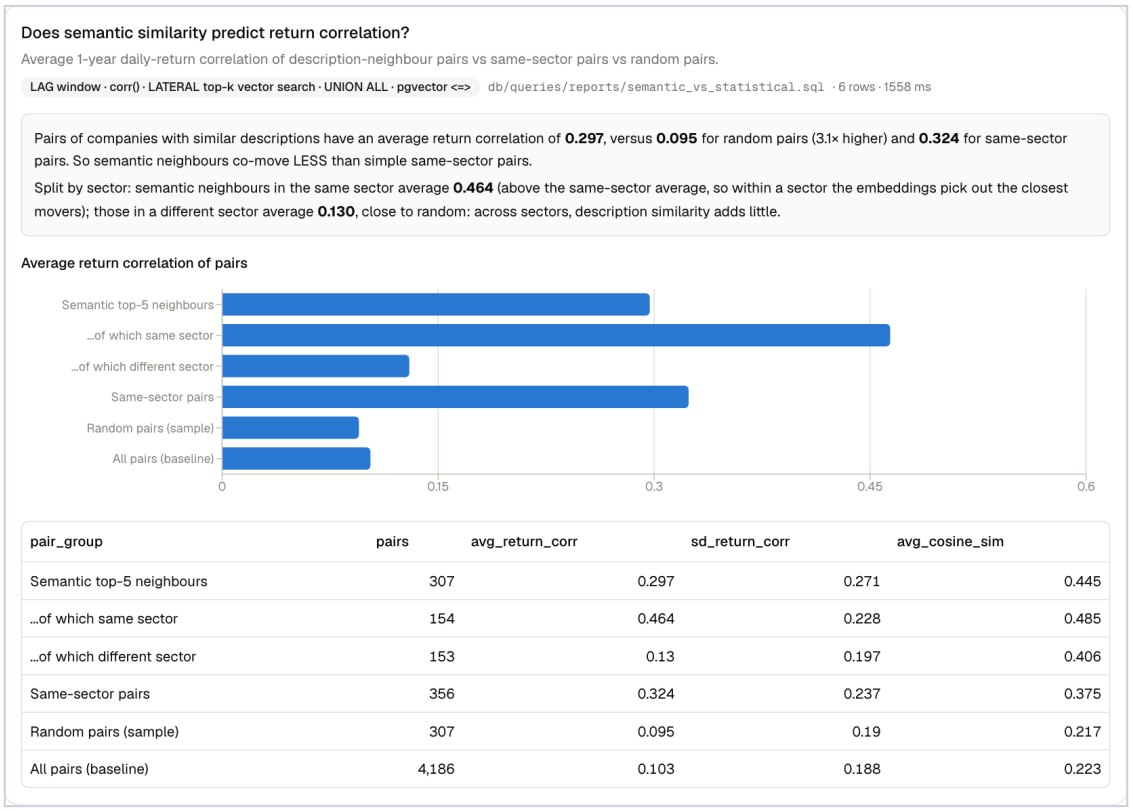


Figure 10.2: Semantic vs statistical similarity for JPM (analyst view, with the "Add JPM to watchlist" button).



sector	companies	avg_pct_change_1y	worst_company_pct	best_company_pct
Energy	7	68.04	50.74	114.17
Healthcare	9	34.74	-14.03	73.92
Basic Materials	7	28.95	-1.36	57.76
Technology	12	21.46	-51.32	117.45
Consumer Defensive	8	15.27	-3.3	38.13
Industrials	9	15.19	-7.05	41.57
Financial Services	10	3.73	-16.77	20.67

Figure 10.3: Reports page, GROUP BY + HAVING report with its parameter form.



pair_group	pairs	avg_return_corr	sd_return_corr	avg_cosine_sim
Semantic top-5 neighbours	307	0.297	0.271	0.445
...of which same sector	154	0.464	0.228	0.485
...of which different sector	153	0.13	0.197	0.406
Same-sector pairs	356	0.324	0.237	0.375
Random pairs (sample)	307	0.095	0.19	0.217
All pairs (baseline)	4,186	0.103	0.188	0.223

Figure 10.4: Reports page, "Does semantic similarity predict return correlation?"

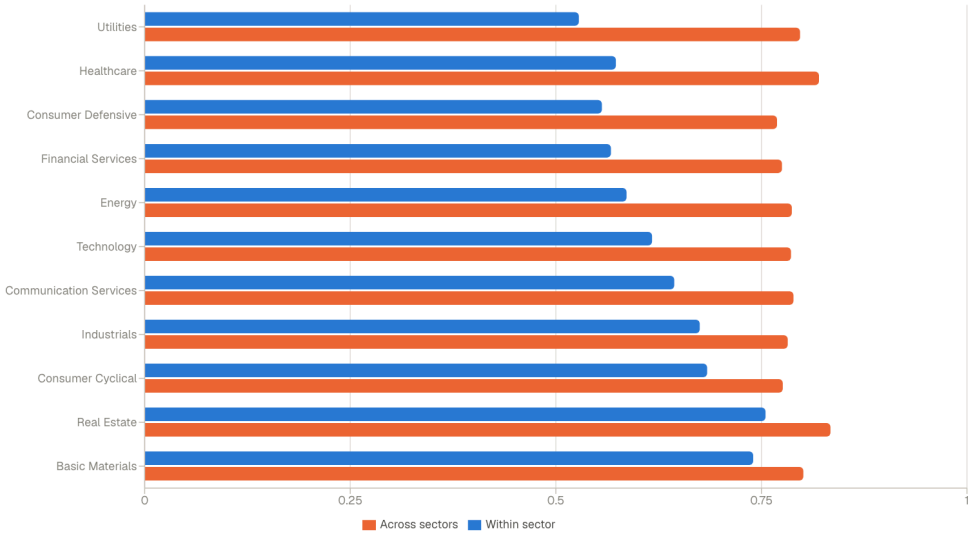
Vector report: do embeddings capture sectors?

Per sector, average embedding distance to same-sector companies vs to other sectors (embeddings never saw the sector label).

`pgvector cosine distance <=> · self-JOIN · AVG with FILTER` `db/queries/reports/sector_embedding_distance.sql · 11 rows · 274 ms`

In **11 of 11** sectors, companies' descriptions are closer (smaller cosine distance) to their own sector than to other sectors, although the model never saw the sector label. Higher ratio = sharper sector clustering.

Average cosine distance (lower = more similar descriptions)



sector	companies	within_sector_distance	cross_sector_distance	ratio_cross_to_within
Utilities	6	0.528	0.797	1.51
Healthcare	9	0.573	0.82	1.43
Consumer Defensive	8	0.556	0.769	1.38
Financial Services	10	0.567	0.775	1.37
Energy	7	0.586	0.787	1.34
Technology	12	0.617	0.786	1.27
Communication Services	8	0.644	0.789	1.22
Industrials	9	0.675	0.782	1.16
Consumer Cyclical	10	0.684	0.776	1.14
Real Estate	6	0.755	0.834	1.1
Basic Materials	7	0.74	0.801	1.08

Figure 10.5: Reports page, vector report: embedding distance within vs across sectors.

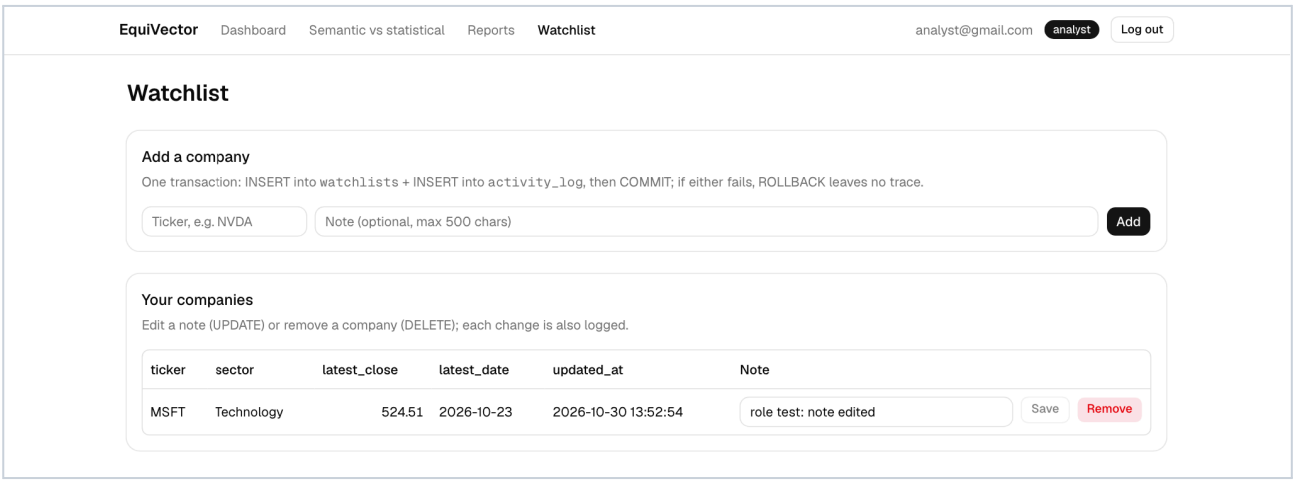


Figure 10.6: Watchlist as analyst: add form, note edit (UPDATE) and remove (DELETE).

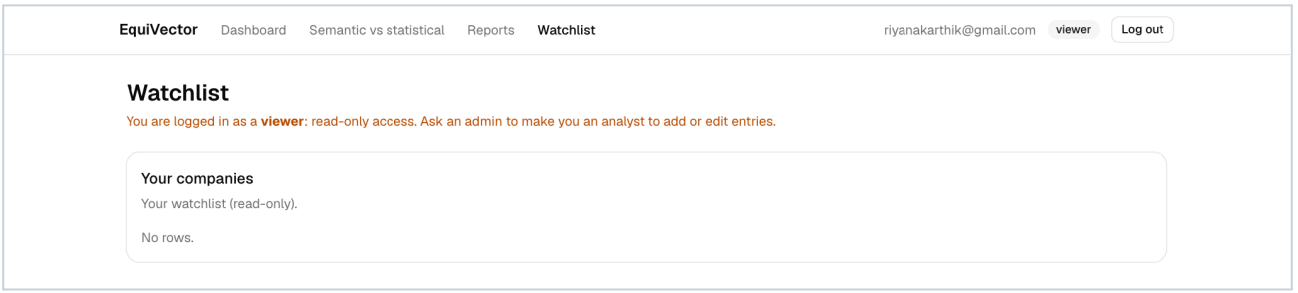


Figure 10.7: Watchlist as viewer: read-only notice, no add/edit/remove controls.

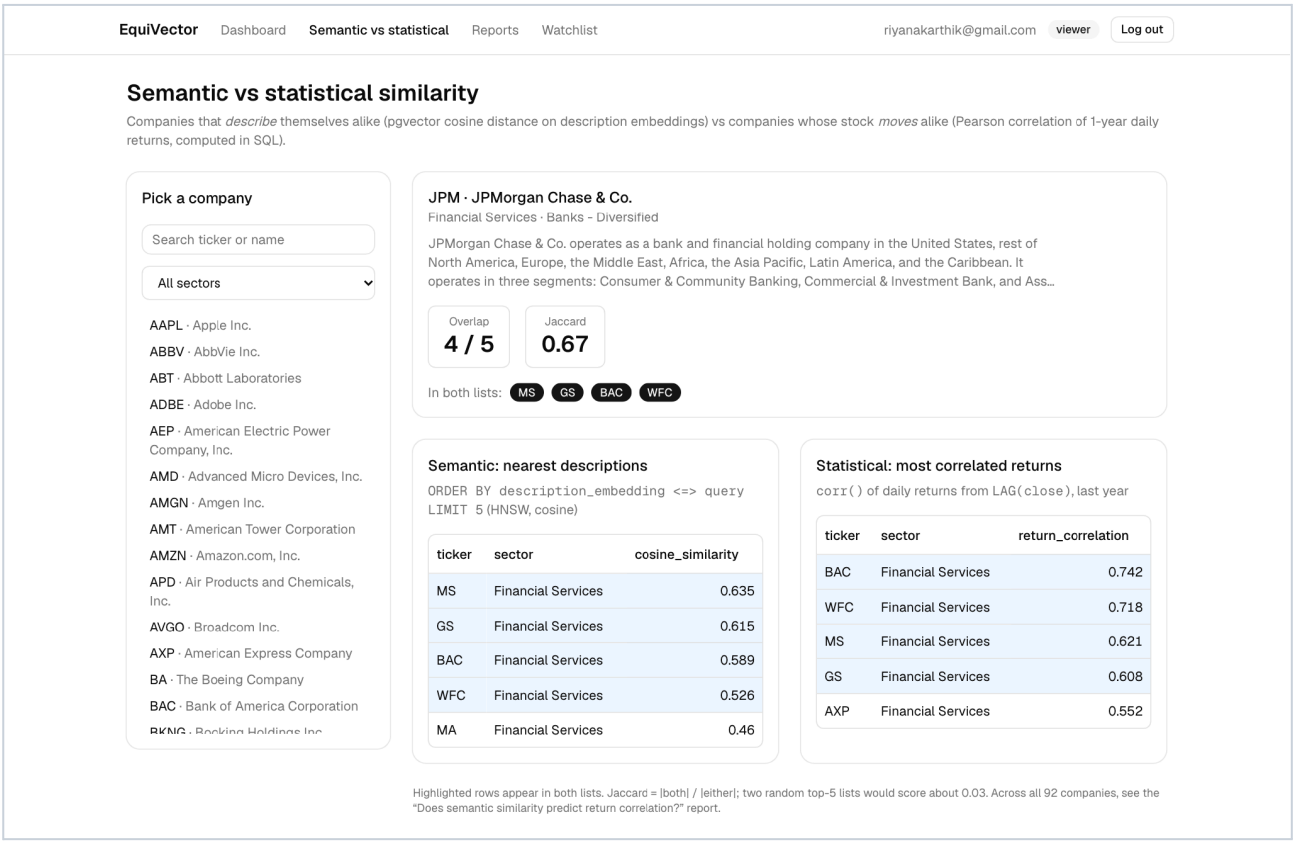


Figure 10.8: Semantic vs statistical as viewer: the "Add to watchlist" button is hidden.

The role is also enforced by the API, not only hidden in the UI. Real response when the viewer session calls the write endpoint directly (captured in the same run):

```
POST /api/watchlist {"ticker":"MSFT"}   (logged in as viewer)
-> 403 {"error":"Analysts only: your role is read-only","role":"viewer"}
```

11. Evaluation Rubrics- Completion Status

#	Criteria	Distribution	Status
1	Database Design	ER diagram , relational schema with keys, relationships, and constraints	**Done**
2	Database Creation	SQL database implementation - DDL with constraints , NoSQL/Vector database creation , Technology justification	**Done**
3	SQL Operations and Queries	DML operations , Nested/correlated/Recursive SQL queries, Join SQL queries	**Partial**
4	Advanced Database Concepts	Advanced database concepts (Procedure/function, trigger, CTE) , NoSQL/Vector feature RUD	**Partial**
5	Web Application and Database Integration	Next.js interface , Backend/API and database integration	**Done**
6	Authentication and Authorization	Authentication implementation (front end and backend) and minimum two roles and role-based access control in database	**Done**
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Documentation, Git/GitHub , Originality and individual contribution	**Partial**

Status checked against the repository on 2026-11-09. Reasons for every Partial:

- **3. SQL Operations and Queries: Partial.** DML, nested and correlated subqueries and joins are done (Section 6). There is **no recursive query** (`WITH RECURSIVE`) anywhere in the project.
- **4. Advanced Database Concepts: Partial.** Done: 3 triggers with 3 PL/pgSQL trigger functions, CTEs, a view, B-tree and HNSW indexes, transactions. Missing: **no stored procedure** (`CREATE PROCEDURE`) and no non-trigger function. **Vector RUD:** Read is done in the app (similarity search); vectors are created and updated only by the seed script `embed_descriptions.py`; there is no delete operation for vectors apart from deleting the company row.
- **7. Documentation, Git/GitHub, Demonstration, Testing, Originality: Partial.** Documentation (`docs/`), tests with recorded outputs (`docs/evidence/`) and a GitHub remote exist. However, all 6 commits have a single author on a single branch (`main`), so individual

contributions are not visible in Git. **[PLACEHOLDER: individual contribution of each team member]**